

LAB MANUAL

WEB PROGRAMMING

Course Code: CSWP-265



**DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF ENGINEERING & COMPUTER SCIENCES
NATIONAL UNIVERSITY OF MODERN LANGUAGES
H-9, ISLAMABAD
PAKISTAN**

Preface

To Apply the knowledge to understand the design and to management of web site design and development to solve the real world problems. Gain knowledge and learning advance web technologies. Be successfully accepted in future's development scenario as an engineering graduate pursuit of lifelong learning. This lab is about front-end and back-end web programming. In this lab students will learn about latest web programming technologies and tools from basic and they will also learn about the concept and usage of rest full APIs to manage and maintain client-side functionalities

Objective

- To enhance students' programming skills and train them in the domain of Visual Programming.
- To provide necessary skills to the students for developing professional software applications using state of the art tools and technologies

Tools/ Technologies

Following tools and technologies will be used for implementing software programs in lab:

1. HTML
2. CSS
3. JavaScript
4. MySQL
5. PHP
6. Laravel
7. React
8. Templating engine
9. XAMP
10. Visual studio code

Effectiveness

This course will develop an ability to design and implement static and dynamic website. The student will be able to choose best technologies for solving web client/server problems. The student will understand, analyze and build web applications using state of the art technologies and tools.

Weekly Breakdown

Week No.	Title of the Lab
1	Creating development environment setup
2	Learning the following HTML tags for web pages required for front-end. • HTML5 page structure • HTML headings • Anchor link, • Line break • Paragraph • Image • Div • Ordered and unordered list tags
3	Learn to create tables using HTML • Designing registration form using HTML
4	Designing first page of website using HTML and CSS • Design the navigation bar using CSS property display: flex • Enhance the cosmetic look of HTML table using CSS. • Design a contact form by customizing HTML form with input controls • Add alignment of form: In center vertically and horizontally regard less of screen size. • Design complete layout of web page using HTML, CSS and CSS Grid.

5	Create a PHP MySQL CRUD application using MySQLi.
6	Create an Object Oriented Programming based PHP MySQL CRUD application using PDOs.
7	Create a PHP session based user authentication and authorization system also make sure to upload user photo at the time of registration.
8	Mid-term evaluation
9	• Validate a HTML form using JavaScript and display appropriate messages using DOM manipulation. • Use fetch to display album title in an unordered list from the API https://jsonplaceholder.typicode.com/albums.
10	• Design responsive layout for a shopping catalog using Bootstrap grid, card, and helper classes. • Design a pop-up contact us form using Bootstrap form controls, modal, and helper classes. • Use Bootstrap carousel with controls, indicators and captions to display a company's portfolio in shape of images and text on them.
11	Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine (Part 1).
12	Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine (Part 2).
13	Create RESTful APIs for tasks table having columns id, text, day and reminder using Laravel.
14	Consuming GET API to display data in React application
15	Using POST and DELETE APIs in React application
16	Final-term evaluation

TABLE OF CONTENTS

Preface.....	2
Objective	2
Tools/ Technologies	2
Effectiveness	3
Weekly Breakdown.....	3
LAB 1: Creating development environment setup	7
LAB 2: Create an HTML file and use HTML5 page structure, headings, anchor link, line break, paragraph, image, div, ordered and unordered list tags.	15
LAB 3: Learn to create tables using HTML. Designing registration form using HTML.	20
LAB 4: Designing first page of website using HTML and CSS	28
LAB 5: Create a PHP MySQL CRUD application using MySQLi	33
LAB 6: Create an Object-Oriented Programming based PHP MySQL CRUD application using PDOs.	42
LAB 7: A PHP session based user authentication and authorization system.....	53
LAB 8: Mid-term Evaluation.....	69
LAB 9: HTML form validation using JavaScript.....	69
LAB 10: Responsive layout design, contact us pop-ups and carousel design using bootstrap.....	73
LAB 11: Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine (Part 1)	78
LAB 12: Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine (Part 2)	86
LAB 13: Create RESTful APIs for tasks table having columns id, text, day and reminder using Laravel.....	93
LAB 14: Consuming GET API to display data in React application	107
LAB 15: Using POST and DELETE APIs in React application.....	116
LAB 16: Final-term Evaluation	122

LAB 1: Creating development environment setup

Objectives:

- Setting up PHP development environment
- Installing XAMPP, VS Code

Theoretical Description:

What is PHP?

PHP (Hypertext PreProcessor) is a general-purpose programming language developed by Rasmus Lerdorf in 1994 for web development. This is one of the most popular programming language for beginners in web development because of its simplicity, large community and accessibility.

Lab Task(s):

Task 1: Steps to setup PHP environment on a local machine: There are basically two ways to set up PHP on a local machine which are:

- Manually install all the required packages (MySQL, PHP & Apache) and configure them.
- An easy and almost error-free method to install PHP on a local machine which is by using all in one package called XAMPP.

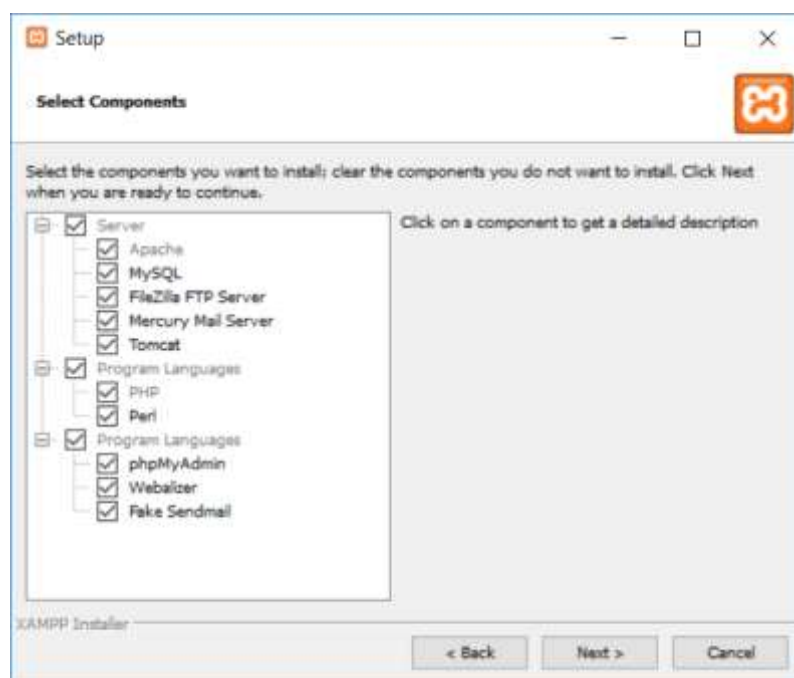
Task 2: Steps to Installing XAMPP:

- Downloading XAMPP: You can download the XAMPP software from the official website here with the latest windows version and latest PHP version.
- Open the downloaded .exe file: After opening the downloaded file you will see a popup from windows, click yes and proceed further.

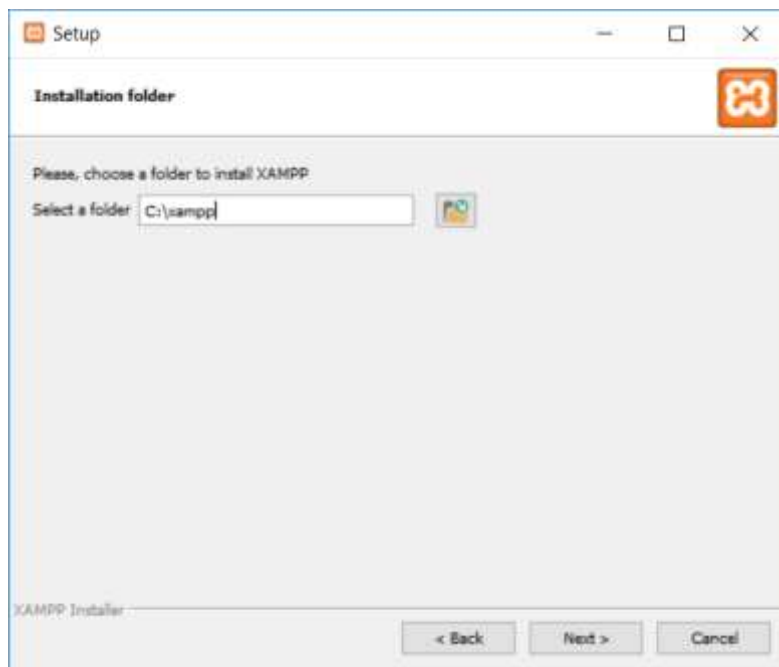
- Click on 'next': You'll see a welcome window of XAMPP like below, click next.



- Select the components: Select the components you want to install. Please select the MySQL and phpMyAdmin components, all other components are optional for this tutorial.



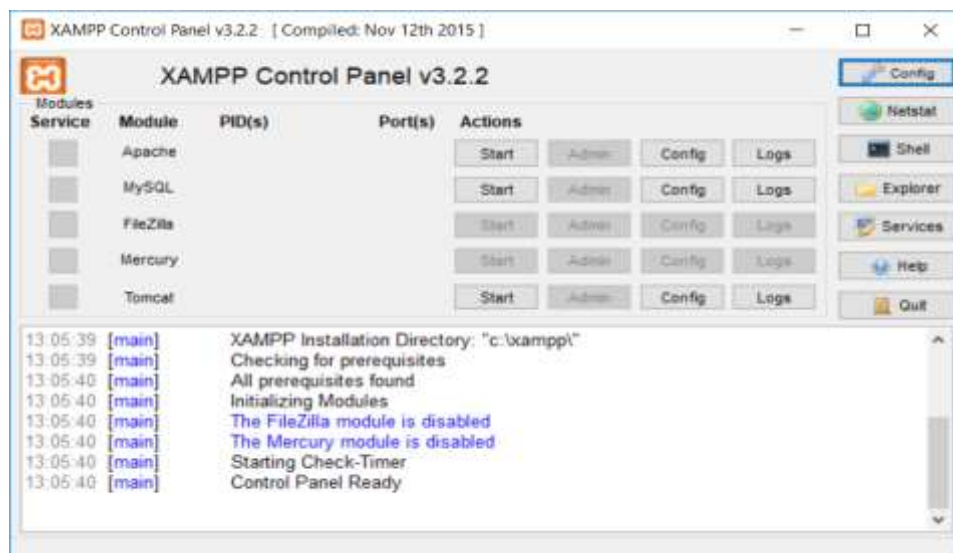
- **Select the installation location:** Select the location you want to install the XAMPP, the default is **C:\xampp**.



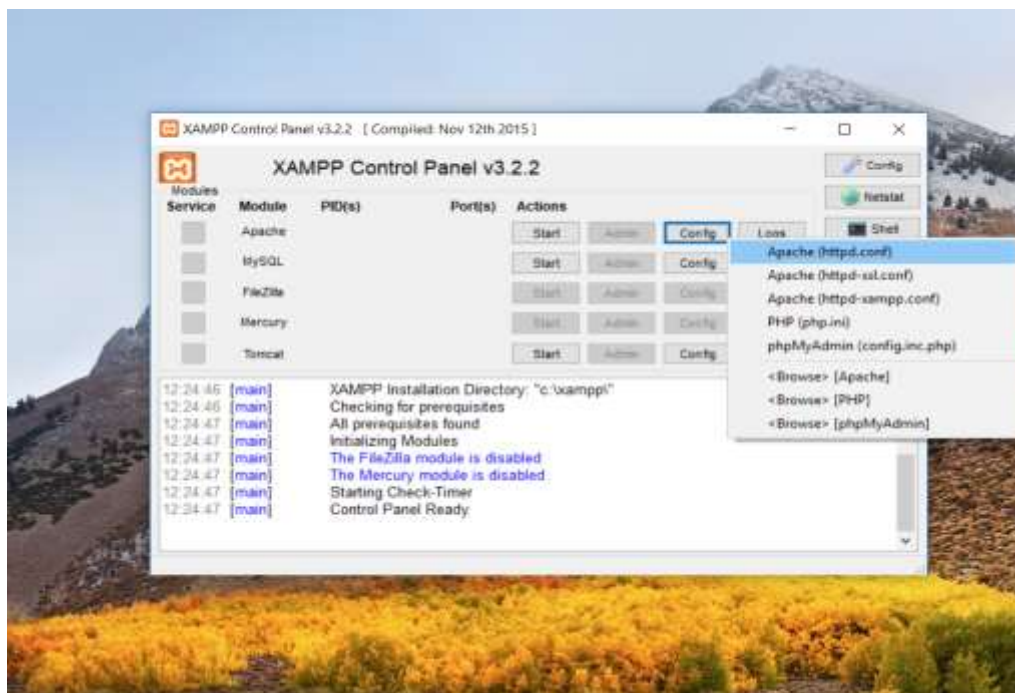
- Click next: **Your clicking next, your installation will begin.**



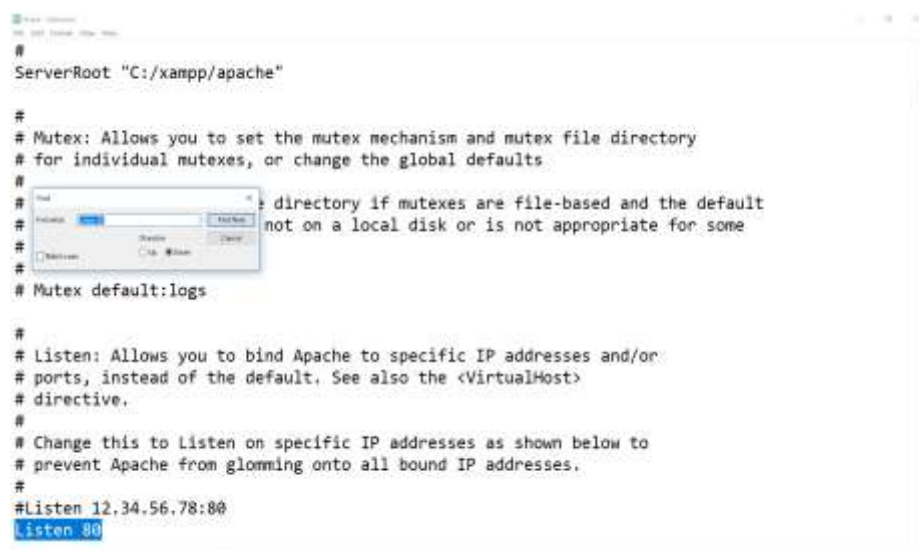
- **Open XAMPP control panel:** After successfully installing the XAMPP in your local machine open the control panel by searching in the windows search bar for 'XAMPP control panel' or by going to the installation directory of XAMPP. You'll see a window like below.



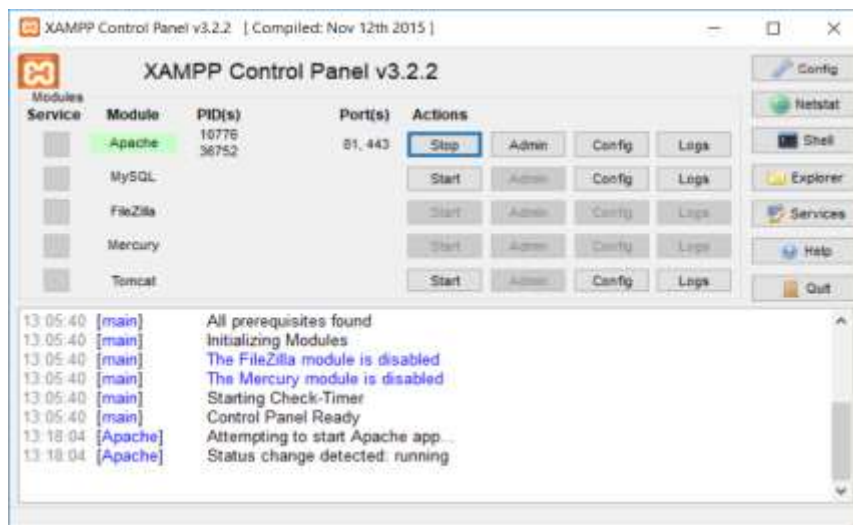
- **Configuring Apache*:** This step is optional i.e. if you are having some issues on windows 10 related to blocked port. Open the configuration file for Apache as shown in the image below.



- Press CTRL + F and search for LISTEN 80. Replace the port **80** with something like **81** and save the file and restart control panel.



- **Start the Apache server:** Start the Apache server by clicking the **start** button you'll see a port number just in front of the Apache column. You can stop the service whenever you want and start any service by just clicking the start button.



- **Checking the installation:** Go to your browser and type **localhost:81** (or simply **localhost** in case you haven't changed the port). You'll see a page like below.



- **Checking PHP installation:** Create a **php** file in htdocs folder in your installation directory. (C:/XAMPP/htdocs) and add the following code in it.

```
<?php
    echo phpinfo();
?>
```

- Save the file as **demo.php** and go to your browser and type localhost:81/demo.php (or simply **localhost/demo.php** in case you haven't changed the port). If the installation goes fine you'll see a result like below.



- If all the above things have gone fine then the environment is successfully set up a PHP development environment in your local machine.

Task3: Installing XAMPP, VS Code + Environment Setup:

VS Code

- Go to the link <https://code.visualstudio.com/download> to download VS Code for your respective operating system and platform.
- Open the downloaded setup file and install it. The installation process is pretty simple and self-explanatory.
- The next step is to install the XAMPP software. It provides interpreters for scripts written in the PHP, along with server solutions.

XAMPP

- Go to this link to download XAMPP according to your OS and platform.
- Open the downloaded setup file. Follow the setup wizard and install XAMPP.
- Run XAMPP after the installation. Start MySQL and Apache server by clicking Start in front of it. (This has been practically done and explained in the video)

Note: If you face any issue while starting the servers in step 3 regarding Port 80, make sure port 80 is not being used by any other application such as Skype. You can check the port usage stats in the Resource Monitor of the windows OS.

Open a web browser, type localhost/phpMyAdmin in the address bar, and hit enter. The phpMyAdmin page will open, proving that the installed software and other things are working right.

LAB 2: Create an HTML file and use HTML5 page structure, headings, anchor link, line break, paragraph, image, div, ordered and unordered list tags.

Objectives:

- To understand the use of HTML.
- To learn element of web-page.

Theoretical Description:

Create an HTML file and use HTML5 page structure, headings, anchor link, line break, paragraph, image, div, ordered and unordered list tags.

Lab Task(s):

TASK 1: Create basic HTML/HTML5 Elements.

Solution:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <header>
    <h1>Welcome to My Page</h1>
  </header>
  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">About</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
  </nav>
  <main>
    <h2>About Me</h2>
    <p>I am a web developer who loves to create beautiful and functional websites.</p>
    <br>
    
    <div>
      <h3>Skills</h3>
      <ul>
```

```
<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>
</ul>
</div>
<h3>Experience</h3>
<ol>
  <li>Web Developer, ABC Company (2018-2020)</li>
  <li>Front-End Developer, XYZ Company (2020-Present)</li>
</ol>
</main>
<footer>
  <p>Copyright 2021</p>
</footer>
</body>
</html>
```

Code Explanations:

- The **doctype** declaration specifies that this is an HTML5 document.
- The **html** element wraps the entire page.
- The **head** element contains the page title, which is displayed in the browser tab.
- The **body** element contains the content of the page, including the **header**, **nav**, **main**, and **footer sections**.
- The **header** element contains the main heading of the page.
- The **ul** element is used to create a list of items that are not in a specific order. Each list item is wrapped in a **li** element.
-
- The **ol** element is used to create a numbered list of items, with each list item being wrapped in a **li** element.
- The **nav** element contains a navigation menu, which is created using an unordered list and anchor links.
- The **main** element contains the main content of the page, including a paragraph, an image, a **div** element with an unordered list, and an ordered list.
- The **footer** element contains a copyright notice.

TASK 2: Create the following tables:

A	B	C	D
E	F	G	H
I	J	K	L
M	N	O	P

Solution:

```
<!DOCTYPE html>
<head>
  <meta charset="utf-8">
  <title>Tables Introduction</title>
  <style>
    table,th,td{
      border: 1px solid black;
      border-collapse: collapse;
      padding: 15px;
    }
  </style>
</head>
<body>
<table>
  <tr>
    <td>A</td>
    <td>B</td>
    <td>C</td>
    <td>D</td>
  </tr>
  <tr>
    <td>E</td>
    <td>F</td>
    <td>G</td>
    <td>H</td>
  </tr>
  <tr>
    <td>I</td>
    <td>J</td>
    <td>K</td>
    <td>L</td>
  </tr>
  <tr>
    <td>M</td>
    <td>N</td>
    <td>O</td>
    <td>P</td>
  </tr>
</table>
```

```
</table>
</body>
```

Code Explanations:

The table element in HTML is used to create a table. It consists of rows, which are represented by the **tr** element, and cells, which are represented by the **td** element. The **th** element is used to create table headers, which are cells that typically appear at the top of a table and contain the names of the columns.

TASK 3:

S.No.	Roll Number	Name	Gender
1	2011	Ali	Male
2	2012	Fatima	Female
3	2013	Asad	Male
4	2014	Asif	Male
5	2015	Maryam	Female

Solution:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
  <style>
    table,th,td{
      border: 1px solid black;
      border-collapse: collapse;
      padding: 15px;
    }
  </style>
</head>
<body>
  <table>
    <thead>
      <tr>
        <th>S.No.</th>
        <th>Roll Number</th>
        <th>Name</th>
        <th>Gender</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>1</td>
```

```
<td>2011</td>
<td>Ali</td>
<td>Male</td>
</tr>
<tr>
<td>2</td>
<td>2012</td>
<td>Fatima</td>
<td>Female</td>
</tr>
<tr>
<td>3</td>
<td>2013</td>
<td>Asad</td>
<td>Male</td>
</tr>
<tr>
<td>4</td>
<td>2014</td>
<td>Asif</td>
<td>Male</td>
</tr>
<tr>
<td>5</td>
<td>2015</td>
<td>Maryam</td>
<td>Female</td>
</tr>
</tbody>
</table>
</body>
```

Code Explanations:

The table element in HTML is used to create a table. It consists of rows, which are represented by the **tr** element, and cells, which are represented by the **td** element. The **th** element is used to create table headers, which are cells that typically appear at the top of a table and contain the names of the columns.

LAB 3: Learn to create tables using HTML. Designing registration form using HTML.

Objectives:

- To understand the use of tables HTML.
- To design form using HTML.

Theoretical Description:

Creation of tables, merging rows, merging columns and the basic layout of the table.

Lab Task(s):

TASK 1: Create the following tables:

A	B		C
D	E	F	G
H	I	J	
K		L	M
		N	O

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Table in Html</title>
  <style>
table, th, td {
  border: 2px solid black;
  border-collapse: collapse;
  padding: 15px;
}
</style>
```

```
</head>
<body>
  <table>

    <tr>
      <td>A</td>
      <td colspan="2">B</td>
      <td>C</td>

    </tr>

    <tr>
      <td>D</td>
      <td>E</td>
      <td>F</td>
      <td rowspan="2">G</td>

    </tr>

    <tr>
      <td>H</td>
      <td>I</td>
      <td>J</td>

    </tr>

    <tr>
      <td rowspan="2" colspan="2">K</td>
      <td>L</td>
      <td>M</td>

    </tr>

    <tr>

      <td>N</td>
      <td>O</td>

    </tr>

  </table>
</body>
```

</html>

Code Explanation:

The code starts with a <html> tag and then has an opening <head> tag. The head section of the code contains meta tags that are used to specify the character encoding, viewport, and title for the page.

The body section is where all of the content goes. It starts with a table element which has three rows and two columns. Each row in this case is called a "tr". In each tr there is one cell that can be identified by its colspan attribute or rowspan attribute. The code is an example of a table in html.

TASK 2:

N=165	ACTUALLY POSITIVE	ACTUALLY NEGATIVE
PREDICTED POSITIVE	50	10
PREDICTED NEGATIVE	5	100

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Table in Html</title>
  <style>
    table, th, td {
      border: 2px solid black;
      border-collapse: collapse;
      padding: 15px;
    }
  </style>
</head>
<body>

  <table>

    <tr>
      <td>N=165</td>
```

```

        <th>Actual Positive</th>
        <th>Actual Negative</th>
    </tr>

    <tr>
        <th>Predicted Positive</th>
        <td>50</td>
        <td>10</td>
    </tr>

    <tr>
        <th>Predicted Negative</th>
        <td>5</td>
        <td>100</td>
    </tr>
</table>

</body>
</html>

```

Code explanation:

The code then has a meta name="viewport" content="width=device-width, initial-scale=1.0"> tag which sets the viewport size of the page to be as wide as possible without being distorted on smaller screens and set at 1x scale (so it will appear in its natural state).

Next comes a title tag followed by some CSS rules that define how table cells should look like when they are displayed on screen: table {border: 2px solid black; bordercollapse: collapse; padding: 15px;} th {border: 2px solid black; bordercollapse: collapse; padding: 15px;} td {border: 2px solid black; bordercollapse: collapse; padding :15px;}

The code is a table, with two columns. The first column has the values of "N=165" and "Actual Positive". The second column has the values of "Predicted Positive" and "Predicted Negative". The code starts off by creating a <table> tag, which contains two rows. Each row contains one <tr> tag that defines the beginning and end of each row.

Task 3:

Registration form

User personal information

Enter your full name

Enter your email

Enter your password

confirm your password

Enter your gender
☐ Male
☐ Female

Select your hobbies
☐ Cricket
☐ Football
☐ Hockey

Enter your Address:

Select Country:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form</title>
```



```

</head>

<body>

    <h1>Registration Form</h1>

    <form action="#" method="post">
        <fieldset>
            <legend>User Personal Information</legend>
            <div>
                <label for="Full_Name">Enter your Full Name</label>
                <br>
                <input type="text" name="fullName" id="Full_Name" required>
            </div>

            <div>
                <label for="Email">Enter your Email</label>
                <br>
                <input type="email" name="email" id="Email" required>
            </div>

            <div>
                <label for="Password">Enter your Password</label>
                <br>
                <input type="password" minlength="6" maxlength="10" name="password"
id="Password" required>
            </div>

            <div>
                <label for="age">Enter your Age</label>
                <br>
                <input type="number" name="age" id="age" min="17" max="65" step="1"
required>
            </div>

            <div>
                <label for="Gender">Select Your Gender:</label>
                <br>
                <input type="radio" name="Gender" value="Male" required> Male
                <br>
                <input type="radio" name="Gender" value="Female" > Female
                <br>
                <input type="radio" name="Gender" value="Other" > Other
                <br>
            </div>
        </fieldset>
    </form>

```

```

</div>

<div>
  <label for="Hobby"> Select Your Hobbies:</label>
  <br>
  <input type="checkbox" name="hobbies[]" value="Cricket" >Cricket
  <br>
  <input type="checkbox" name="hobbies[]" value="Football" >Football
  <br>
  <input type="checkbox" name="hobbies[]" value="Hockey" >Hockey
  <br>
  <input type="checkbox" name="hobbies[]" value="Table_tennis" >Table Tennis
  <br>

</div>

<div>
  <label for="address">Enter Your Address</label>
  <br>
  <textarea name="Address" id="address" cols="20" rows="10"
placeholder="Enter your Address" required></textarea>

</div>

<div>

  <label for="Country">Please Select Your Country</label>
  <br>
  <select name="Country" id="Country" required>
    <option value="Pak" selected>Pakistan</option>
    <option value="Afg">Afghanistan</option>
    <option value="Ind" >India</option>
    <option value="Chi">China</option>
    <option value="Ir">Iran</option>

  </select>
</div>

</fieldset>

<fieldset>
  <legend>Educational Information</legend>

```

```
<div>

  <label for="Education">Please Select Your Higher Education</label>
  <br>
  <select name="Education" id="Education" required>
    <option value="8th">8th</option>
    <option value="Matric">Matriculation</option>
    <option value="Inter" >INTER</option>
    <option value="BS">Bachelor</option>
    <option value="MS" selected>Master</option>
    <option value="PHD">PHD</option>

  </select>
</div>

</fieldset>
<div>
  <input type="submit" value="submit" name="submit">
  <input type="reset" value="reset">
</div>
</form>

</body>

</html>
```

LAB 4: Designing first page of website using HTML and CSS

Objectives:

- To design complete structure of web-page using HTML and CSS with CSS grid.

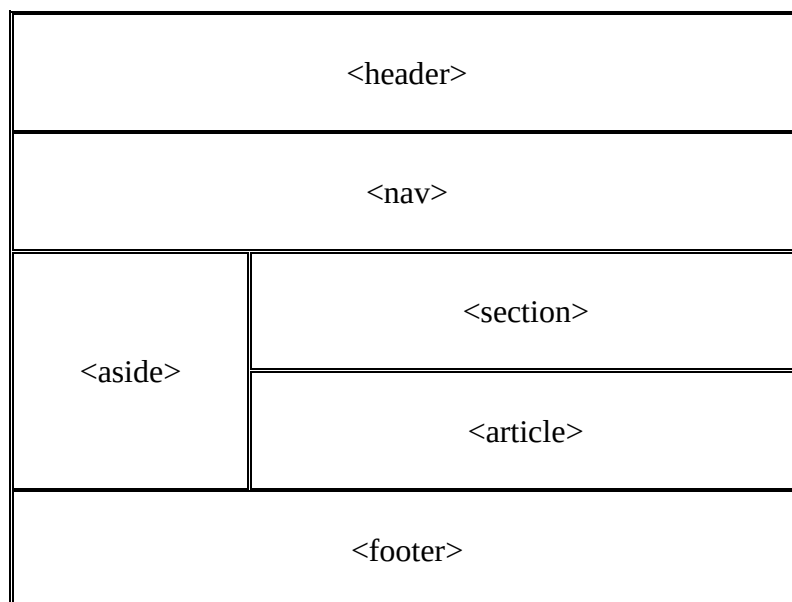
Theoretical Description:

Creating layouts with the CSS Grid

Lab Task(s):

TASK 1

- Design the navigation bar using CSS property display: flex
- Enhance the cosmetic look of HTML table using CSS.
- Design a contact form by customizing HTML form input controls and make sure that form is displayed in center vertically and horizontally regard less of screen size.
- Design the following layout using HTML, CSS and CSS Grid:



Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Home</title>
```

```

<style>
  aside {
    width: 30%;
    padding-left: 15px;
    margin-left: 15px;
    float: right;
    font-style: italic;
    background-color: lightgray;
  }
  nav {
    background-color: lightblue;
    color: white;
    font-family: sans-serif;
    text-align: center;
  }
  nav ul {
    display: flex;
    flex-direction: row;
    justify-content: space-between;
    list-style: none;
    margin: 0;
    padding: 0;
  }
  nav li {
    display: inline-block;
    padding: 20px;
  }
  table, th, td {
    border: 1px solid black;
    border-collapse: collapse;
    padding: 15px;
    background-color: lightblue;
    font-family: sans-serif;
    font-size: 16px;
    color: white;
  }
</style>

</head>
<body>
<header>
  <header id="main-header">
    <div id="company-name">
      Company Int.PVT
    </div>
    <div id="main-nav">
      <nav>
        <a href="#" .class=active>Home</a>
        <a href="#">Services</a>
        <a href="#">About Us</a>
      </nav>
    </div>
  </header>

```

```

        <a href="#">Contact Us</a>
    </nav>
</div>
</header>

<p>My family and I visited The Epcot center this summer. The weather was nice, and
Epcot was amazing! I had a great summer together with my family!</p>
<aside>
    <p>
        The aside tag defines some content aside from the content it is placed in.

        The aside content should be indirectly related to the surrounding content.

        Tip: The aside content is often placed as a sidebar in a document.</p>
</aside>
<section>
    <h2>What is Section?</h2>
    <p>In HTML, the section element is a block-level element that is used to
represent a standalone section of content within a document. It is often used to group related
content together, such as chapters in a book, articles in a newspaper, or sections on a
webpage.
The section element can contain any type of content, including headings, paragraphs, lists,
tables, images, and other block-level elements. It can also be used to apply styles or layout
using CSS, or to add behavior using JavaScript.</p>
<table>
    <tr>
        <td>A</td>
        <td>B</td>
        <td>C</td>
        <td>D</td>
    </tr>
    <tr>
        <td>E</td>
        <td>F</td>
        <td>G</td>
        <td>H</td>
    </tr>
</table>
</section>

    <article>
        <h3>What is article?</h3>
        <p>
            An article should make sense on its own and it should be possible to
            distribute it independently from the rest of the site.
        </p>
        <br>
        <form style="display:flex;
            flex-direction: column; align-items:center; height:100vh; margin:auto;
            width:50%;">
            <label for="name">Name:</label><br>

```

```

        <input type="text" id="name" name="name" style="width:100%;"><br>
        <label for="email">Email:</label><br>
        <input type="email" id="email" name="email" style="width:100%;"><br>
        <label for="subject">Subject:</label><br>
        <input type="text" id="subject" name="subject" style="width:100%;"><br>
        <label for="message">Message:</label><br>
        <textarea id="message" name="message"
style="width:100%;"></textarea><br>
        <button type="submit" style="width:100%;">Send</button>
    </form>
    </article>
<footer>
    <p>Copyright 2023</p>
</footer>
</header>
</body>
</html>

```

Code Explanations:

Header & Navbar:

Create an HTML element for the navigation bar, such as a nav element. Inside the navigation bar, create a list of items using the ul element and list items using the li element. Add the display:flex property to the ul element to make it a flex container. Add the flex-direction:row property to the ul element to make the items appear in a row. Add the justify-content:space-between property to the ul element to distribute the items evenly across the width of the container.

Tables:

Change the background color of the table, rows, or cells: You can use the background-color property to change the background color of the table, rows, or cells. Add borders to the table, rows, or cells: You can use the border property to add borders to the table, rows, or cells. Change the font and text color of the table: You can use the font-family, font-size, and color properties to change the font and text color of the table. Add padding to the cells: You can use the padding property to add padding to the cells.

Forms:

In this example, the form has been customized using CSS styles to set the width of the form to 50% of the screen, and the width of the form controls to 100% of the form width. You can customize the form further by

using additional HTML attributes and CSS styles, such as the placeholder attribute to set a placeholder text for the text fields, or the font-size and color properties to customize the font and text.

LAB 5: Create a PHP MySQL CRUD application using MySQLi

Objectives:

- To learn CURD application using PHP and MySQL and MySQLi.

Theoretical Description:

Using the basic CRUD operations to run queries on the database.

Lab Task(s):

Task 1: Create a PHP MySQL CRUD application using MySQLi.

Product.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product From</title>
  <!-- <link rel="stylesheet" href="/student.css"> -->
  <style> *{
    margin: 0%;
    padding: 0%;
    box-sizing: border-box;
  }
  .container{
    width: 100vw;
    height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    background: rgb(79, 78, 78);
  }
  .form{
    /* display: flex; */
    width: 400px;
    height: auto;
    border: 2px solid red;
    display: flex;
    justify-content: center;
    align-items: center;
    background-color: azure;
    box-shadow: 0px 5px 6px white;
```

```

}
.form label{
    display: flex;
    margin: 10px 4px;
    font-weight: 600;
}
.form input{
    padding: 5px 20px;
    border-radius: 10px;
}
.submit input{
    cursor: pointer;
    margin-top: 5px;
    padding: 9px 25px ;
}
</style>
</head>
<body>
    <div class="container">
        <div class="form">
            <form action="insert.php" method="post">
                <label for="Name">Enter the Name of Product </label>
                <input type="text" name="Name" id="Name" placeholder="Enter the Student Name"
required>

                <label for="brand">Enter the brand of Product</label>
                <input type="text" name="brand" id="brand" placeholder="Enter the brand of
Product" required>

                <label for="Price">Enter the Price of Product</label>
                <input type="number" name="Price" id="Price" placeholder="Enter the Price of
Product" required>

                <label for="Quantity">Enter the Quantity of Product</label>
                <input type="number" name="Quantity" id="Quantity" placeholder="Enter the
Quantity of Product" required>

                <br>
                <div class="submit">

                    <input type="submit" value="Submit Record">
                </div>
            </form>
        </div>
    </div>

```

```
        </form>
    </div>
</div>

</body>
</html>
```

Insert.php

```
<?php

// Connection Establishment

$con = new mysqli('localhost', 'root', '', 'productdb');

if($con->connect_error){
    die("Error". $con->connect_error);
}

// Making Variable

$name= $_POST['Name'];
$brand= $_POST['brand'];
$price= $_POST['Price'];
$quantity= $_POST['Quantity'];

//Writing the Query

$query = " insert into products(Name, brand, Price, Quantity)
values ('".$name."', '".$brand."', '".$price."', '".$quantity."')";

//Executing the Query
```

```
$res=$con->query($qry);

if($res){
    echo " Data is Inserted";
}
else{
    echo "Not Inserted";
}

// Connection Close

$con->close();
?>
```

Display.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Display</title>
</head>
<body>

<a href="products.html">Insert New Record</a>

</body>
</html>

<?php

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "productdb";

// Create connection
```

```
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error)

{
    die("Connection failed: " . $conn->connect_error);
}
$qry = "SELECT * from students";
$result = $conn->query($qry);

if ($result->num_rows > 0) {
    // output data of each row

    echo "<table border = 1><tr>";
    echo"<th> Name </th> ";
    echo"<th>brand</th> ";
    echo"<th>Price </th>";
    echo "<th>Quantity</th>" ;

    while($row = $result->fetch_assoc()) {
        echo "<tr><td>".$row["Name"]."</td>";

        echo "<td>" . $row["brand"]. "</td>";

        echo "<td>" . $row["Price"]. "</td>";
        echo "<td>" . $row["Quantity"]. "</td>";

        echo "<td><a href='UpdateForm.php? P_ID=".$row['P_ID']."'> Update </a> | <a
href='delete.php? P_ID=".$row['P_ID']."'> Delete </a></td>";

        echo "</tr><br>";
    }
    echo "</table>";
} else {
    echo "0 results";
}
$conn->close();
?>
```

Delete.php

```
<?php
if(!isset($_GET['P_ID'])){
    header('location: Display.php');
}
$host = "localhost";
$dbName = "productdb";
$dbUser = "root";
$dbUserPassword= "";

//conection establish
$con = new mysqli($host,$dbUser,$dbUserPassword,$dbName);
if($con->connect_error){

    die("Error: ".$con->connect_error);
}

$qry = "DELETE FROM products where P_ID =".$_GET['P_ID']."";
$res=$con->query($qry);

if($res){
    echo "Data DELETED";
}
else{
    echo "Data not Deleted";
}
echo "<br><a href= 'Display.php'>Back to HOME</a>";
$con->close();

?>
```

Update.php

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "productdb";

$conn = new mysqli($servername,$username,$password,$dbname);
if($conn->connect_error){
    die("Connection error." . $conn->connect_error);
}
```

```

$S_ID= $_POST['P_ID'];
$Name= $_POST['Name'];
$brand= $_POST['brand'];
$Price= $_POST['Price'];
$Quantity= $_POST['Quantity'];

$qry = "UPDATE productss
SET Name='".$Name."', brand= '".$brand."', Price='".$Price."',
Quantity='".$Quantity.'"");

$result = $conn->query($qry);
if($result){
    echo'<script>alert("Successfully Updated")</script>';
}
else{
    echo"Error";
}
$conn->close();
?>

```

updateFrom.php

```

<?php
if(!isset($_GET['P_ID'])){
    header('location: Display.php');
}

$host = "localhost";
$dbName = "productdb";
$dbUser = "root";
$dbUserPassword= "";

//connection establish
$con = new mysqli($host,$dbUser,$dbUserPassword,$dbName);
if($con->connect_error){

    die("Error: ".$con->connect_error);
}

$qry = "SELECT * FROM students where P_ID='".$_GET['P_ID']."'";

```

```

$res =$con->query($qry);

if($res->num_rows > 0)
{
    $row = $res->fetch_assoc();

    $P_ID = $row['P_ID'];
    $Name = $row['Name'];
    $brand=$row['brand'];
    $Price=$row['Price'];
    $Quantity=$row['Quantity'];

} else
{
    echo "Error while executing query:<br>".$qry." Error:<br>".$con-
>connect_error;
}

echo "Product ID:".$P_ID;

$con->close();

?>
<!DOCTYPE html>
<head>
    <title>Update-UserForm</title>
</head>
<body>

    <form action="update.php" method="post">
<br>
        <label for="Name">Enter Product Name:</label>
        <br>
        <input type="text" id="Name" name="Name" value="<?php echo
isset($row['Name'])? $row['Name']: '' ?>">
        <br><br>
        <label for="brand">Enter the brand Name</label>
        <br>
        <input type="text" id="brand" name="brand" value="<?php echo
isset($row['brand'])? $row['brand']: '' ?>">
        <br><br>
        <label for="Price">Enter The Product Price:</label>

```



```

        <br>
        <input type="number" id="Price" name="Price" value="<?php echo
isset($row['Price'])? $row['Price']: '' ?>">
        <br><br>
        <label for="Quantity">Enter the Product Quantity:</label>
        <br>
        <input type="number" id="Quantity" name="Quantity" value="<?php echo
isset($row['Quantity'])? $row['Quantity']: '' ?>">

        <td><input type="hidden" name="P_ID" id="P_ID" value="<?php echo
isset($row['P_ID'])? $row['P_ID']: '' ?>"></td>

        <input type="submit" value="UPDATE" name="save">
        <br>
        <button><a href="Display.php">Back TO HOME PAGE</a> </button>

        <br>
        <button><a href="products.html">Add New User</a></button>

    </form>
</body>

```

Code Explanation:

In this code we have done the Curd using the Mysqli in which we have create the following file

- **Product.html** which is used to create the form to get the data from the users.
- **Insert.php** which is use to insert the data in the database.
- **Display.php** is use to show the existence data in the database.
- **Delete.php** is used to delete the existence data from the database.
- **Update.php** is used to update the existence data from the database.

LAB 6: Create an Object-Oriented Programming based PHP MySQL CRUD application using PDOs.

Objectives:

To learn to develop CURD application using PHP and MySQL with the concept of OOP.

Theoretical Description:

Using the OOP based CRUD operations to run queries on the database.

Lab Task(s):

TASK 1 : Create an Object Oriented Programming based PHP MySQL CRUD application using PDOs.

Solution:

Db.config.php File:

```
<?php
define ("host","localhost");
define ("DbUser","root"); //constant define kisa ha  pela is ka key ha or dusra Argument ha is
ka
define ("DBPassword","");
define ("dbname","db");
class DataBase{
    private $pdo;
    public function __construct()
    { //object 3 argument lata hai
        //DNS driver server DBName
        try{
            $this->pdo =new PDO('mysql:host=localhost;dbname=db',DbUser,DBPassword);
            $this->pdo->setAttribute(PDO::ATTR_ERRMODE,PDO::ERRMODE_EXCEPTION);
            //echo "connected";
        }
        catch(PDOException $e){
            die("Error".$e);
        }
    }
    public function UDI($qry)
    {
        return $this->pdo->exec($qry); }
    public function Search($qry,$classRef)
    {
        $result = $this->pdo->query($qry);
        if($result->rowCount()>0)
        {
            return $result->fetchAll(PDO::FETCH_CLASS, $classRef);
        }
    }
}
```

```

    }
    return null;
}
}
?>

```

Product.html:

```

<!DOCTYPE html>
<html>
  <head>
    <title>UserForm</title>
  </head>
  <body>

    <form action="productform.php" method="post">

      <label for="ProductName">Enter Product Name:</label>
      <br>
      <input type="ProductName" id="ProductName" name="ProductName">
      <br><br>
      <label for="Size">Enter Product Size:</label>
      <br>
      <input type="Size" id="Size" name="Size">
      <br><br>
      <label for="color">Enter Product Color:</label>
      <br>
      <input type="text" id="color" name="color">
      <br><br>
      <label for="ConstructionMatrrial">Enter Product Construction Matrrial:</label>
      <br>
      <input type="ConstructionMatrrial" id="ConstructionMatrrial" name="ConstructionMatrrial">
      <br><br>
      <label for="Weight">Enter Product Weight:</label>
      <br>
      <input type="Weight" id="Weight" name="Weight">
      <br><br>
      <label for="Features">Enter Product Features:</label>
      <br>
      <input type="Features" id="Features" name="Features">
      <br><br>
      <label for="CountryOrign">Enter Product Country Orign:</label>
      <br>
      <input type="CountryOrign" id="Size" name="CountryOrign">
      <br><br>
      <label for="Price">Enter Product Price:</label>
      <br>
      <input type="Price" id="Size" name="Price">
      <br>
    </form>
  </body>
</html>

```

```

        <td><input type="hidden" name="P_ID" id="P_ID"></td>
        <br>
        <input type="submit" value="submit" name="submit">
        <input type="reset" value="reset" name="reset">

    </form>
</body>
</html>

```

Product.php

```

<?php
$ProductName = $_POST['ProductName'];
$Size=$_POST['Size'];
$color=$_POST['color'];
$ConstructionMatrrial=$_POST['ConstructionMatrrial'];
$Weight=$_POST['Weight'];
$Features=$_POST['Features'];
$CountryOrign=$_POST['CountryOrign'];
$Price=$_POST['Price'];

// $P_ID=$_POST['P_ID'];

//query
$qry = "insert into product
(ProductName,Size,color,ConstructionMatrrial,Weight,Features,CountryOrign,Price)
Values
('".$ProductName."','".$Size."','".$color."','".$ConstructionMatrrial."','".$Weight."','".$Features."','".$CountryOrign."','".$Price."')";

require_once('db.config.php');
$db = new DataBase();

if($db->UDI($qry)){
    echo "data saved";
} else {
    echo "data not saved";
}
?>

```

Index.php:

```

<!DOCTYPE html>
<head>

```

```

<title>Table</title>
<style>
    table{
        border: 1px solid black ;
        padding: 10px;
        inline-size: border-box;
    }
    th{
        border: 1px solid black ;
        padding: 10px;
    }
    td{
        border: 1px solid black ;
        padding: 10px;
    }
</style>

</head>
<body>
<a href="productform.html">Add New Product</a>
<!-- <a href="logout.php">logout</a> -->

<table>
    <tr>
    <th>Product ID</th>
        <th>Product Name</th>
        <th>Size</th>
        <th>color</th>
        <th>Construction Matrial</th>
        <th>Weight</th>
        <th>Product Features</th>
        <th>Orign of Country</th>
        <th>Price</th>
        <th>Actions</th>
    </tr>

<?php

require_once('db.config.php');
require_once('User.php');
$db = new DataBase();
$qry = "SELECT * FROM product";

$result = $db->Search($qry, User::class);

if($result!=null){
    foreach($result as $user){
        echo $user->getDisplayData();
    }
}

```

```

}
else{
    echo "404";
}
?>
</table>
</body>
</html>

```

Delete.php:

```

<?php
if(!isset($_GET['P_ID'])){
    header('location: index.php');
}

require_once('db.config.php');

$db = new DataBase();

$qry = "DELETE FROM product where P_ID =".$_GET['P_ID']."";
$result=$db->UDI($qry);

if($result){
    echo "Data DELETED";
}
else{
    echo "Data not Deleted";
}
echo "<br><a href= 'index.php'>Back to HOME</a>";

?>

```

Update.php:

```

<?php
if(!isset($_GET['P_ID'])){
    header('location: index.php');
}
$ProductName = $_POST['ProductName'];
$Size=$_POST['Size'];
$color=$_POST['color'];
$ConstructionMatrrial=$_POST['ConstructionMatrrial'];
$Weight=$_POST['Weight'];
$Features=$_POST['Features'];
$CountryOrign=$_POST['CountryOrign'];

```

```

$Price=$_POST['Price'];
$P_ID=$_POST['P_ID'];
require_once('db.config.php');
$db = new DataBase();
$qry ="UPDATE product set
ProductName='".$ProductName."',Size='".$Size."',color='".$color."',ConstructionMatrial='".$Cons
tructionMatrial."',Weight='".$Weight."',Features='".$Features."',CountryOrign='".$CountryOrign.
"',Price='".$Price.'" Where P_ID='".$P_ID.'";
if($db->UDI($qry)){
    echo "Data Updated";
}
else{
    echo "Data not Updated";
}
echo "<br><a href= 'index.php'>Back to HOME</a>";
$con->close();
?>

```

User.php:

```

<?php
class User
{
    private $P_ID;
    private $ID;
    private $ProductName;
    private $Size;
    private $color;
    private $ConstructionMatrial;
    private $Weight;
    private $Features;
    private $CountryOrign;
    private $Price;
    // private $name;
    // private $password;

    /**
     * @return mixed
     */
    function getP_ID() {
        return $this->P_ID;
    }

    /**
     * @param mixed $P_ID
     * @return User
     */
    function setP_ID($P_ID) {
        $this->P_ID = $P_ID;
        return $this;
    }
}

```

```
/**
 * @return mixed
 */
function getProductName() {
    return $this->ProductName;
}
/**
 * @param mixed $ProductName
 * @return User
 */
function setProductName($ProductName) {
    $this->ProductName = $ProductName;
    return $this;
}
/**
 * @return mixed
 */
function getSize() {
    return $this->Size;
}
/**
 * @param mixed $Size
 * @return User
 */
function setSize($Size) {
    $this->Size = $Size;
    return $this;
}
/**
 * @return mixed
 */
function getColor() {
    return $this->color;
}
/**
 * @param mixed $color
 * @return User
 */
function setColor($color) {
    $this->color = $color;
    return $this;
}
/**
 * @return mixed
 */
function getConstructionMatrrial() {
    return $this->ConstructionMatrrial;
}
/**
 * @param mixed $ConstructionMatrrial
```



```

    * @return User
    */
function setConstructionMatrrial($ConstructionMatrrial) {
    $this->ConstructionMatrrial = $ConstructionMatrrial;
    return $this;
}
/**
 * @return mixed
 */
function getWeight() {
    return $this->Weight;
}
/**
 * @param mixed $Weight
 * @return User
 */
function setWeight($Weight) {
    $this->Weight = $Weight;
    return $this;
}
/**
 * @return mixed
 */
function getFeatures() {
    return $this->Features;
}
/**
 * @param mixed $Features
 * @return User
 */
function setFeatures($Features) {
    $this->Features = $Features;
    return $this;
}
/**
 * @return mixed
 */
function getCountryOrign() {
    return $this->CountryOrign;
}
/**
 * @param mixed $CountryOrign
 * @return User
 */
function setCountryOrign($CountryOrign) {
    $this->CountryOrign = $CountryOrign;
    return $this;
}
}
/**

```

```

        * @return mixed
        */
function getPrice() {
    return $this->Price;
}

/**
 * @param mixed $Price
 * @return User
 */
function setPrice($Price) {
    $this->Price = $Price;
    return $this;
}

public function getDisplayData()
{
    return "<tr>
        <td>".$this->getP_ID()."</td>
        <td>".$this->getProductName()."</td>
        <td>".$this->getSize()."</td>
        <td>".$this->getColor()."</td>
        <td>".$this->getConstructionMatrrial()."</td>
        <td>".$this->getWeight()."</td>
        <td>".$this->getFeatures()."</td>
        <td>".$this->getCountryOrign()."</td>
        <td>".$this->getPrice()."</td>
        <td>
            <a href='updateForm.php?P_ID=".$this->getP_ID()."'>Update</a>
            |
            <a href='delete.php?P_ID=".$this->getP_ID()."'>Delelte</a>
        </td>
    </tr>";
}
}
?>

```

Code Explanations:

This code defines a DataBase class in PHP that can be used to connect to a MySQL database and perform various database operations. The class has three methods:

1. `__construct()`: This method is used to create a new PDO (PHP Data Object) connection to the database when an object of the DataBase class is created. The connection is made using the PDO class, which is provided by PHP to access databases using PHP. The method sets the error mode to

`PDO::ERRMODE_EXCEPTION`, which causes any errors that occur during database operations to throw an exception.

2. `UDI()`: This method stands for "Update, Delete, Insert" and is used to execute queries that perform these types of operations on the database. The method takes a single argument, which is the query to be executed, and returns the number of affected rows.
3. `Search()`: This method is used to execute queries that retrieve data from the database. The method takes two arguments: the query to be executed and the name of the class to use when creating objects to store the result rows. The method returns an array of objects, with each object representing a row in the result set. If the result set is empty, the method returns null.

The `DataBase` class also defines several constants at the top of the code: `host`, `DbUser`, `DBPassword`, and `dbname`. These constants are used to store the connection details for the database, such as the hostname, username, password, and database name. The constants are used in the `__construct()` method to create the connection to the database.

This PHP code is a simple script that performs a `DELETE` operation on a database table. It first checks if the `'P_ID'` parameter is set in the GET request. If it is not set, it redirects the user to the index page.

The script then includes the `'db.config.php'` file, which is a file that contains code for connecting to the database. It creates an instance of the `'DataBase'` class, which is responsible for interacting with the database. The script then constructs a `DELETE` query using the `'P_ID'` value from the GET request. It executes the query using the `'UDI'` method of the `'DataBase'` class, which stands for 'update, delete, or insert'. If the query is successful, it prints a message saying that the data was deleted. Otherwise, it prints a message saying that the data was not deleted. Finally, it provides a link to the index page.

This is an HTML code that generates a table to display a list of products. The table has 10 columns: "Product ID", "Product Name", "Size", "Color", "Construction Material", "Weight", "Product Features", "Origin of Country", "Price", and "Actions". The table is created using the `table` HTML element, and the rows and columns are created using the `tr` and `td` elements respectively. The first row, which contains the column headings, is created using the `th` element.

The table is populated with data from a database by making a database query using PHP. The `SELECT *` `FROM product` query retrieves all rows from the product table. The resulting data is then iterated through

using a foreach loop, and the `getDisplayData` method of the `User` class is called on each row to generate the HTML code for the corresponding table row.

The table also has a link to "Add New Product" at the top, which leads to an HTML form for adding a new product to the database.

This is a PHP script that is used to insert data into a database table called "product". The script receives the data that needs to be inserted through form inputs. The form inputs are accessed using the `$_POST` superglobal array in PHP. The script creates a connection to the database using the `DataBase` class and then prepares and executes an `INSERT` query using the `UDI()` method of the `DataBase` class. The `INSERT` query inserts the data received from the form into the "product" table. If the query is successful, the script echoes "data saved", otherwise it echoes "data not saved".

LAB 7: A PHP session based user authentication and authorization system.**Objectives:**

- To develop user authentication while creating PHP sessions
- To maintain web-page security by adding user authorization

Theoretical Description:

Create a user authentication example with PHP sessions.

Lab Task(s):

Task 1: Create a PHP session-based user authentication and authorization system also make sure to upload user photo at the time of registration.

Db.config.php

```
<?php
define ("host","localhost");
define ("DbUser","root"); //constant define kiewa ha pela is ka key ha or dusra Argument
ha is ka
define ("DBPassword","");
define ("dbname","productdb");

class DataBase{
    private $pdo;
    public function __construct()
    {
        //object 3 argument lata hai
        //DNS driver server DBName
        try{
            $this->pdo =new
PDO('mysql:host=localhost;dbname=productdb',DbUser,DBPassword);
            $this->pdo->setAttribute(PDO::ATTR_ERRMODE,PDO::ERRMODE_EXCEPTION);
            //echo "connected";
        }

        catch(PDOException $e){
            die("Error".$e);
        }
    }
}
```

```

public function UDI($qry)
{
    return $this->pdo->exec($qry);
}

public function Search($qry,$classRef)
{
    $result = $this->pdo->query($qry);
    if($result->rowCount()>0)
    {
        return $result->fetchAll(PDO::FETCH_CLASS, $classRef);
    }
    return null;
}
}
?>

```

Users.php

```

<?php
class User
{
    private $P_ID;
    private $ID;
    private $ProductName;
    private $Size;
    private $color;
    private $ConstructionMatrial;
    private $Weight;
    private $Features;
    private $CountryOrign;
    private $Price;
    private $name;
    private $password;

    /**
     * @return mixed
     */
    function getP_ID() {
        return $this->P_ID;
    }

    /**
     * @param mixed $P_ID
     * @return User
     */

```

```
function setP_ID($P_ID) {
    $this->P_ID = $P_ID;
    return $this;
}
/**
 * @return mixed
 */
function getProductName() {
    return $this->ProductName;
}
/**
 * @param mixed $ProductName
 * @return User
 */
function setProductName($ProductName) {
    $this->ProductName = $ProductName;
    return $this;
}
/**
 * @return mixed
 */
function getSize() {
    return $this->Size;
}
/**
 * @param mixed $Size
 * @return User
 */
function setSize($Size) {
    $this->Size = $Size;
    return $this;
}
/**
 * @return mixed
 */
function getColor() {
    return $this->color;
}
/**
 * @param mixed $color
 * @return User
 */
function setColor($color) {
    $this->color = $color;
    return $this;
}
/**
```

```

    * @return mixed
    */
function getConstructionMatrial() {
    return $this->ConstructionMatrial;
}
/**
 * @param mixed $ConstructionMatrial
 * @return User
 */
function setConstructionMatrial($ConstructionMatrial) {
    $this->ConstructionMatrial = $ConstructionMatrial;
    return $this;
}
/**
 * @return mixed
 */
function getWeight() {
    return $this->Weight;
}
/**
 * @param mixed $Weight
 * @return User
 */
function setWeight($Weight) {
    $this->Weight = $Weight;
    return $this;
}
/**
 * @return mixed
 */
function getFeatures() {
    return $this->Features;
}
/**
 * @param mixed $Features
 * @return User
 */
function setFeatures($Features) {
    $this->Features = $Features;
    return $this;
}
/**
 * @return mixed
 */
function getCountryOrign() {
    return $this->CountryOrign;
}

```



```

/**
 * @param mixed $CountryOrign
 * @return User
 */
function setCountryOrign($CountryOrign) {
    $this->CountryOrign = $CountryOrign;
    return $this;
}

/**
 * @return mixed
 */
function getPrice() {
    return $this->Price;
}

/**
 * @param mixed $Price
 * @return User
 */
function setPrice($Price) {
    $this->Price = $Price;
    return $this;
}
public function getDisplayData()
{
    return "<tr>
        <td>".$this->getP_ID()."</td>
        <td>".$this->getProductName()."</td>
        <td>".$this->getSize()."</td>
        <td>".$this->getColor()."</td>
        <td>".$this->getConstructionMatrrial()."</td>
        <td>".$this->getWeight()."</td>
        <td>".$this->getFeatures()."</td>
        <td>".$this->getCountryOrign()."</td>
        <td>".$this->getPrice()."</td>
        <td>
            <a href='updateForm.php?P_ID=".$this->getP_ID()." '>Update</a>
            |
            <a href='delete.php?P_ID=".$this->getP_ID()." '>Delelte</a>
        </td>
    </tr>";
}

/**
 * @return mixed
 */

```

```
function getName() {
    return $this->name;
}

/**
 * @param mixed $name
 * @return User
 */
function setName($name): self {
    $this->name = $name;
    return $this;
}

/**
 * @return mixed
 */
function getPassword() {
    return $this->password;
}

/**
 * @param mixed $password
 * @return User
 */
function setPassword($password): self {
    $this->password = $password;
    return $this;
}

/**
 * @return mixed
 */
function getID() {
    return $this->ID;
}

/**
 * @param mixed $ID
 * @return User
 */
function setID($ID): self {
    $this->ID = $ID;
    return $this;
}
}
?>
```

Products.html

```

<!DOCTYPE html>
<head>
    <title>UserForm</title>
</head>
<body>

    <form action="productform.php" method="post">

        <label for="ProductName">Enter Product Name:</label>
        <br>
        <input type="ProductName" id="ProductName" name="ProductName">
        <br><br>
        <label for="Size">Enter Product Size:</label>
        <br>
        <input type="Size" id="Size" name="Size">
        <br><br>
        <label for="color">Enter Product Color:</label>
        <br>
        <input type="text" id="color" name="color">
        <br><br>
        <label for="ConstructionMatrrial">Enter Product Construction Matrrial:</label>
        <br>
        <input type="ConstructionMatrrial" id="ConstructionMatrrial"
name="ConstructionMatrrial">
        <br><br>
        <label for="Weight">Enter Product Weight:</label>
        <br>
        <input type="Weight" id="Weight" name="Weight">
        <br><br>
        <label for="Features">Enter Product Features:</label>
        <br>
        <input type="Features" id="Features" name="Features">
        <br><br>
        <label for="CountryOrign">Enter Product Country Orign:</label>
        <br>
        <input type="CountryOrign" id="Size" name="CountryOrign">
        <br><br>
        <label for="Price">Enter Product Price:</label>
        <br>
        <input type="Price" id="Size" name="Price">
        <br>

        <td><input type="hidden" name="P_ID" id="P_ID"></td>
        <br>
        <input type="submit" value="submit" name="submit">
        <input type="reset" value="reset" name="reset">

```

```

    </form>
</body>

```

Productsform.php

```

<?php

if(isset( $_SESSION['ID'])){
    header("location:SignIn.php");
}
?>
<?php

$ProductName = $_POST['ProductName'];

$Size=$_POST['Size'];
$color=$_POST['color'];
$ConstructionMatrrial=$_POST['ConstructionMatrrial'];
$Weight=$_POST['Weight'];
$Features=$_POST['Features'];
$CountryOrign=$_POST['CountryOrign'];
$Price=$_POST['Price'];

// $P_ID=$_POST['P_ID'];

//query

$qry = "insert into product
(ProductName,Size,color,ConstructionMatrrial,Weight,Features,CountryOrign,Price)
Values
('".$ProductName."','".$Size."','".$color."','".$ConstructionMatrrial."','".$Weight."','".$
Features."','".$CountryOrign."','".$Price."')";

require_once('db.config.php');
$db = new DataBase();

if($db->UDI($qry)){
    echo "data saved";
} else {
    echo "data not saved";
}

?>

```

Display.php:

```

<?php
session_start();
if(!isset( $_SESSION['ID'])){
    header("location:SignIn.php");
}
?>

<!DOCTYPE html>
<head>
    <title>Table</title>
</head>
<body>
<a href="productform.html">Add New User</a>
<a href="logout.php">logout</a>

<table>
    <tr>
        <th>Product ID</th>
        <th>Product Name</th>
        <th>Size</th>
        <th>color</th>
        <th>Construction Matrial</th>
        <th>Weight</th>
        <th>Product Features</th>
        <th>Orign of Country</th>
        <th>Price</th>
        <th>Actions</th>
    </tr>

<?php

require_once('db.config.php');
require_once('User.php');
$db = new DataBase();
$qry = "SELECT * FROM product";

$result = $db->Search($qry, User::class);

if($result!=null){
    foreach($result as $user){
        echo $user->getDisplayData();
    }
}

```

```

}
else{
    echo "404";
}
?>
</table>
</body>
</html>

```

Delete.php

```

<?php
if(!isset($_GET['P_ID'])){
    header('location: index.php');
}

require_once('db.config.php');

$db = new DataBase();

$qry = "DELETE FROM product where P_ID =".$_GET['P_ID']."";
$result=$db->UDI($qry);

if($result){
    echo "Data DELETED";
}
else{
    echo "Data not Deleted";
}
echo "<br><a href= 'index.php'>Back to HOME</a>";

?>

```

Update.php

```

<?php
if(!isset($_GET['P_ID'])){
    header('location: index.php');
}

$productName = $_POST['ProductName'];

```

```

$Size=$_POST['Size'];
$color=$_POST['color'];
$ConstructionMatrial=$_POST['ConstructionMatrial'];
$Weight=$_POST['Weight'];
$Features=$_POST['Features'];
$CountryOrign=$_POST['CountryOrign'];
$Price=$_POST['Price'];

$P_ID=$_POST['P_ID'];

require_once('db.config.php');

$db = new DataBase();

$qry ="UPDATE product set
ProductName='".$ProductName."',Size='".$Size."',color='".$color."',ConstructionMatrial='".$
$ConstructionMatrial."',Weight='".$Weight."',Features='".$Features."',CountryOrign='".$Cou
ntryOrign."',Price='".$Price.'" Where P_ID='".$P_ID.'";

if($db->UDI($qry)){
    echo "Data Updated";
}
else{
    echo "Data not Updated";
}
echo "<br><a href= 'index.php'>Back to HOME</a>";
$con->close();
?>

```

updateFrom.php

```

<?php

if(!isset($_GET['P_ID'])){
    header('location: index.php');
}

$qry = "SELECT * FROM product where P_ID = '".$_GET['P_ID']."'";
require_once('db.config.php');
require_once('User.php');
$db = new DataBase();
$result = $db->Search($qry, User::class);

```

```

if($result==null){
    header('location: index.php');
}
if($result!=null){
    // echo "Data";

$row = $result[0];
//print_r($row);

$P_ID = $row->getP_ID();
$ProductName = $row->getProductName();
$Size=$row->getSize();
$color=$row->getColor();
$ConstructionMatrial=$row->getConstructionMatrial();
$Weight=$row->getWeight();
$Features=$row->getFeatures();
$CountryOrign=$row->getCountryOrign();
$Price=$row->getPrice();
}

?>
<!DOCTYPE html>
<head>
    <title>Update-UserForm</title>
</head>
<body>

    <form action="Update.php" method="post">
<br>
        <label for="ProductName">Enter Product Name:</label>
        <br>
        <input type="text" id="ProductName" name="ProductName" value="<?php echo
$row->getProductName() ?>">
        <br><br>
        <label for="Size">Enter Product Size:</label>
        <br>
        <input type="text" id="Size" name="Size" value="<?php echo $row->getSize() ?>">
        <br><br>
        <label for="color">Enter Product Color:</label>
        <br>
        <input type="text" id="color" name="color" value="<?php echo $row->getColor() ?>">
        <br><br>
        <label for="ConstructionMatrial">Enter Product Construction Matrial:</label>
        <br>

```



```

        <input type="ConstructionMatrial" id="ConstructionMatrial"
name="ConstructionMatrial" value="<?php echo $row->getConstructionMatrial() ?>">
        <br><br>
        <label for="Weight">Enter Product Weight:</label>
        <br>
        <input type="Weight" id="Weight" name="Weight" value="<?php echo $row->getWeight()
?>">

        <br><br>
        <label for="Features">Enter Product Features:</label>
        <br>
        <input type="Features" id="Features" name="Features" value="<?php echo $row-
>getFeatures() ?>">
        <br><br>
        <label for="CountryOrign">Enter Product Country Orign:</label>
        <br>
        <input type="CountryOrign" id="Size" name="CountryOrign" value="<?php echo $row-
>getCountryOrign() ?>">
        <br><br>
        <label for="Price">Enter Product Price:</label>
        <br>
        <input type="Price" id="Size" name="Price" value="<?php echo $row->getPrice() ?>">
        <br>

        <td><input type="hidden" name="P_ID" id="P_ID" value=<?php echo $row-
>getP_ID()?></td>

        <input type="submit" value="UPDATE" name="save">
        <br>
        <button><a href="index.php">Back</a> </button>

        <br>
        <button><a href="productform.html">Add New User</a></button>

    </form>
</body>

```

Registration.html

```

<!DOCTYPE html>
<head>
    <title>Registration-Form</title>
</head>
<body>

    <form action="Registration.php" method="post">

```

```

<label for="name">Enter UserName:</label>
<br>
<input type="name" id="name" name="name">
<br>
<label for="password">Enter Password:</label>
<br>
<input type="password" id="password" name="password">
<br>

<label for="image">Enter Your recent photo:</label>
<br>
<input type="file" accept="image/*" />
<br>
<td><input type="hidden" name="ID" id="ID"></td>
<br>
<input type="submit" value="Save" name="submit">
<input type="reset" value="reset" name="reset">

</form>
</body>

```

Registration.php

```

<?php
$name=$_POST['name'];
$password=$_POST['password'];
$ID=$_POST['ID'];

$qry="insert into user(name,password) values('".$name."','".$password."')";
require_once("db.config.php");
$db = new DataBase();
if($db->UDI($qry)){
    echo "You Registration is Sucessfullly Completed";
}
else{
    echo "You Registration is NOT Sucessfullly Completed";
    echo "Please Try Again!...<a href='Registration.html' > >Back to Registration
Page</a>";
}
echo "<br>Click To
<a href= 'SignIn.php'>SignIn</a>";

?>

```

SignIn.php

```

<?php
session_start();
if(isset( $_SESSION['ID'])) {
    //header("location:SignIn.php");
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>SignIn</title>
</head>
<body>

    <form action="SignIn.php" method="post">

        <label for="name">Enter UserName:</label>
        <br>
        <input type="text" id="name" name="name">
        <br>
        <label for="password">Enter Password:</label>
        <br>
        <input type="password" id="password" name="password">
        <br>
        <td><input type="hidden" name="ID" id="ID"></td>
        <br>
        <input type="submit" value="SigIn" name="submit">

    </form>
</body>
</html>

<?php
if(isset($_POST['submit']))
{
    $name=$_POST['name'];
    $password=$_POST['password'];
    $ID=$_POST['ID'];
    $qry="Select *From user Where name='$name' and password='$password'";
    require_once("db.config.php");
    require_once("User.php");
    $db = new DataBase();
    $result = $db->Search($qry, User::class);
    if($result!=Null)

```

```

    {
        $row=$result[0];
        $_SESSION['ID']=$row->getID();
        header("location:index.php");
        //bad ma change be karo ga
        echo"login Sucessfully";
    }
    else{
        echo"Invalid credentials";
        echo "Please Try Again!...<a href='Registration.html' > >Back to Registration
Page</a>";
    }
}

?>

```

Logout.php

```

<?php
session_start();
if(!isset( $_SESSION['ID'])){
    header("location:SignIn.php");
}
else
{
    session_destroy();
    header("location:SignIn.php");
}
?>

```

Code Explanation:

First we have created the Curd using the PDO and then make a registration form where new users get himself registered and signIn through that username and password and signIn.php is used to verify the user and then login and logout.php is used to terminate the session.

LAB 8: Mid-term Evaluation.

LAB 9: HTML from validation using JavaScript.

Objectives:

- Form validation and display of appropriate messages.
- API use for displaying of album title in unordered list.

Theoretical Description:

Create a user authentication example with PHP sessions.

Lab Task(s):

TASK 1: Validate a HTML form using JavaScript and display appropriate messages using DOM manipulation.

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>validate Form</title>
</head>
<body>
  <form id="form">
    <label for="name">Name:</label><br>
    <input type="text" id="name" ><br>
    <label for="email">Email:</label><br>
    <input type="email" id="email" ><br>
    <label for="password">Password:</label><br>
    <input type="password" id="password" required minlength="8"><br>
    <label for="confirmPassword">Confirm Password:</label><br>
    <input type="password" id="confirmPassword" required minlength="8"><br><br>
    <button type="submit">Submit</button>
  </form>

  <div id="errorMessage"></div>
```

```

<script>
  const form = document.getElementById('form');
  const errorMessage = document.getElementById('errorMessage');

  form.addEventListener('submit', (event) => {
    event.preventDefault(); // prevent the form from submitting

    // clear any previous error messages
    errorMessage.innerHTML = '';

    // validate form fields
    const name = document.getElementById('name').value.trim();
    const email = document.getElementById('email').value.trim();
    const password = document.getElementById('password').value;
    const confirmPassword = document.getElementById('confirmPassword').value;

    if (name === '') {
      errorMessage.innerHTML = 'Name is required';
    } else if (email === '') {
      errorMessage.innerHTML = 'Email is required';
    } else if (password === '') {
      errorMessage.innerHTML = 'Password is required';
    } else if (password !== confirmPassword) {
      errorMessage.innerHTML = 'Passwords do not match';
    } else if (password == confirmPassword) {
      errorMessage.innerHTML = 'Passwords match';
    } else {
      // form is valid, submit form
      //errorMessage.innerHTML = 'Passwords match';
    }
    form.submit();
  });

</script>
</body>
</html>

```

This code defines an HTML form with four input fields (name, email, password, and confirm password) and a submit button. It also includes a div element with an id of errorMessage to display error messages.

The form includes several input elements with various attributes such as required, type, and minlength. These attributes are used to enforce certain rules on the form fields, such as requiring a field to be filled in or requiring a password to be at least 8 characters long.

The form also includes a script tag at the bottom of the page with JavaScript code that listens for the submit event of the form. When the submit button is clicked, the event listener function is executed. Inside the

function, the form submission is prevented using the preventDefault method, and any previous error messages are cleared from the errorMessage element.

Next, the script validates the form fields by checking whether they are empty or whether the password and confirm password fields match. If any errors are found, an appropriate error message is displayed in the errorMessage element. If all form fields are valid, the form is submitted using the submit method.

TASK 2: Use fetch to display album title in an unordered list from the API <https://jsonplaceholder.typicode.com/albums>.

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fetch with JavaScript</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Kadq2F9CUG65" crossorigin="anonymous">
</head>
<body>
  <ul id="albums">

  </ul>
  <script>
    fetch('https://jsonplaceholder.typicode.com/albums')
    .then(response => response.json())
    .then(albums => {
      const ul = document.getElementById('albums');
      albums.forEach(album => {
        const li = document.createElement('li');
        li.textContent = album.title;
        ul.appendChild(li);
      });
    })
    .catch(error => console.error(error));
  </script>
</body>
```

In this example, we use the **fetch** function to make a GET request to the API endpoint at <https://jsonplaceholder.typicode.com/albums>. The **then** method is used to parse the response as JSON and pass the resulting object to the next **then** method.

Inside the second **then** method, we get a reference to the **ul** element using the **getElementById** method and then loop through the **albums** array using the **forEach** method. For each album, we create a new **li** element using the **createElement** method, set its **textContent** to the album title, and then append it to the **ul** element using the **appendChild** method.

If there is an error making the request or parsing the response, it is logged to the console using the **console.error** method.

LAB 10: Responsive layout design, contact us pop-ups and carousel design using bootstrap.

Objectives:

- Creating responsive designs for variety of screens.
- Using bootstrap grid to design pop-up contact us messages and carousel slider.

Theoretical Description:

Create a responsive design with the bootstrap's grid system.

Lab Task(s):

Task 1: Design responsive layout for a shopping catalog using Bootstrap grid, card, and helper classes.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-GLhlTQ8iRABdZLL603oVMWSktQOp6b7In1Zl3/Jr59b6EGGoI1aFkw7cmDA6j6gD" crossorigin="anonymous">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/js/bootstrap.bundle.min.js" integrity="sha384-w76AqPfDkMBDXo30jS1Sgez6pr3x5MlQ1ZAGC+nuZB+EYdgRZgiwxhTBTkF7CXvN" crossorigin="anonymous"></script>
  <title>Document</title>
</head>
<body>

  <div class="container text-center">
    <div class="row">
      <div class="col">
        <div class="card" style="width: 18rem;">
          
          <div class="card-body">
            <h5 class="card-title">T shirt</h5>
            <p class="card-text">Some quick example text to build on the card title and make up the bulk of the card's content.</p>
            <a href="#" class="btn btn-primary">Add to cart</a>
            <a href="#" class="btn btn-success">Buy Now</a>
          </div>
        </div>
      </div>
    </div>
  </div>
```

```

        </div>
    </div>
</div>
<div class="col">
    <div class="card" style="width: 18rem;">
        
        <div class="card-body">
            <h5 class="card-title"> hoodies</h5>
            <p class="card-text">Some quick example text to build on the card title and
make up the bulk of the card's content.</p>
            <a href="#" class="btn btn-primary">Add to Cart</a>
            <a href="#" class="btn btn-success">Buy Now</a>

        </div>
    </div>
</div>
<div class="col">
    <div class="card" style="width: 18rem;">
        
        <div class="card-body">
            <h5 class="card-title">Jeans Pent</h5>
            <p class="card-text">Some quick example text to build on the card title and
make up the bulk of the card's content.</p>
            <a href="#" class="btn btn-primary">Add to Cart</a>
            <a href="#" class="btn btn-success">Buy Now</a>

        </div>
    </div>
</div>
</div>
</div>
</body>
</html>

```

Task 2: Design a pop-up contact us form using Bootstrap form controls, modal, and helper classes.

```

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

```

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-
GLh1TQ8iRABdZLL603oVMWSktQOp6b7In1Zl3/Jr59b6EGGoI1aFkw7cmDA6j6gD" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-
w76AqPfDkMBDXo30jS1Sgez6pr3x5MlQ1ZAGC+nuZB+EYdgRZgiwxhTBTkF7CXvN"
    crossorigin="anonymous"></script>
<title>Pop up Contact</title>
</head>

<body>
    <!-- <button type="button" class="btn btn-primary" data-toggle="modal" data-
target="#myModal">Contact Us</button> -->

    <div class="mb-3">
        <label for="exampleFormControlInput1" class="form-label">Name</label>
        <input type="text" class="form-control" id="exampleFormControlInput1"
placeholder="name@example.com">
    </div>
    <div class="mb-3">
        <label for="exampleFormControlInput1" class="form-label">Email address</label>
        <input type="email" class="form-control" id="exampleFormControlInput1"
placeholder="name@example.com">
    </div>
    <div class="mb-3">
        <label for="exampleFormControlTextarea1" class="form-label"> textarea</label>
        <textarea class="form-control" id="exampleFormControlTextarea1"
rows="3"></textarea>
    </div>

    <button type="button" class="btn btn-primary">Submit</button>
    <button type="button" class="btn btn-danger">Reset</button>
</body>
</html>

```

Task 3: Use Bootstrap carousel with controls, indicators and captions to display a company's portfolio in shape of images and text on them.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Bootstrap Example</title>
    <meta charset="utf-8">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.1/jquery.min.js"></script>
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>
</head>
<body>

<div class="container">
  <h2>Carousel Example</h2>
  <div id="myCarousel" class="carousel slide" data-ride="carousel">
    <!-- Indicators -->
    <ol class="carousel-indicators">
      <li data-target="#myCarousel" data-slide-to="0" class="active"></li>
      <li data-target="#myCarousel" data-slide-to="1"></li>
      <li data-target="#myCarousel" data-slide-to="2"></li>
    </ol>

    <!-- Wrapper for slides -->
    <div class="carousel-inner">
      <div class="item active">
        
        <!-- Caption -->
        <div class="carousel-caption" style="color: black;">
          <h3>Wear Cloths that Matters</h3>
          <p>The clothing we are buying is something that supports our personal values.</p>
        </div>
      </div>

      <div class="item">
        
        <div class="carousel-caption" style="color: black;">
          <h3>Wear Cloths that Matters</h3>
          <p>The clothing we are buying is something that supports our personal values.</p>
        </div>
      </div>

      <div class="item">
        
        <div class="carousel-caption" style="color: whitesmoke;">
          <h3>Wear Cloths that Matters</h3>
          <p>The clothing we are buying is something that supports our personal values.</p>
        </div>
      </div>
    </div>

    <!-- Left and right controls -->
    <a class="left carousel-control" href="#myCarousel" data-slide="prev">
      <span class="glyphicon glyphicon-chevron-left"></span>
      <span class="sr-only">Previous</span>
    </a>
  </div>

```

```
</a>
<a class="right carousel-control" href="#myCarousel" data-slide="next">
  <span class="glyphicon glyphicon-chevron-right"></span>
  <span class="sr-only">Next</span>
</a>
</div>
</div>

</body>
</html>
```

Code Explanations:

This code is an example of a Bootstrap carousel, which is a slideshow that displays a series of images or other content. It is implemented using HTML, CSS, and JavaScript.

The carousel is contained within a div element with a class of "container". It has an ID of "myCarousel" and uses the "carousel" and "slide" classes from Bootstrap. The "data-ride" attribute is set to "carousel", which causes the carousel to start automatically when the page loads.

The carousel has indicators, which are represented by an ordered list (ol) element with a class of "carousel-indicators". Each list item (li) represents a slide and has a "data-target" attribute that points to the ID of the carousel and a "data-slide-to" attribute that indicates the index of the slide. The first list item has an additional class of "active", which indicates that it is the initial slide that is displayed.

The slides are contained within a div element with a class of "carousel-inner". Each slide is represented by a div element with a class of "item". The first slide also has an additional class of "active", which indicates that it is the initial slide that is displayed.

Each slide has an image element that displays an image. The carousel also has captions, which are contained within a div element with a class of "carousel-caption".

The carousel has left and right controls, which are represented by anchor (a) elements with classes of "left carousel-control" and "right carousel-control", respectively. The "href" attribute of the anchor elements points to the ID of the carousel, and the "data-slide" attribute indicates which direction to slide when the control is clicked. The controls include span elements with classes of "glyphicon glyphicon-chevron-left" and "glyphicon glyphicon-chevron-right" to display left and right arrows, respectively. The "sr-only" class hides the controls from sighted users and provides an indication to screen reader users.

LAB 11: Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine (Part 1)

Objectives:

- Understanding the use of Laravel to create CURD application.
- Learning Blade templating engine.

Theoretical Description:

Create a responsive design with the bootstrap's grid system.

Lab Task(s):

Step 1: Install Laravel 9

First of all make sure that you have installed PHP dependency manager Composer then, we need to get fresh Laravel 9 version application using bellow command, so open your terminal OR command prompt and run bellow command:

composer create-project --prefer-dist laravel/laravel customer

Migrations:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
use Illuminate\Validation\Rules\Unique;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('customers', function (Blueprint $table) {
            $table->id();
            $table->string('name',150);
            $table->string('cnic',15);
            $table->string('gender',15);
```

```

        $table->string('phone',15);

        $table->timestamps();
    });

}

/**
 * Reverse the migrations.
 *
 * @return void
 */
public function down()
{
    Schema::dropIfExists('customers');
}

};

```

This is a Laravel migration file. It defines a class with up and down methods, which are used to create and delete the customers table in a database, respectively.

The up method uses the Schema facade to create a new table called customers, which has an id field as a primary key and an auto-incrementing integer, a name field as a string with a maximum length of 150 characters, a cnic field as a string with a maximum length of 15 characters, a gender field as a string with a maximum length of 15 characters, a phone field as a string with a maximum length of 15 characters, and timestamps fields for created_at and updated_at timestamps.

The down method uses the Schema facade to drop the customers table if it exists. When this migration is run using the php artisan migrate command, the customers table will be created in the database with the specified fields and column types. If the migration is rolled back using the php artisan migrate:rollback command, the customers table will be deleted from the database.

Models:

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Customer extends Model
{

```

```

use HasFactory;
protected $fillable=[
    'name','cnic','gender','phone',
];
}

```

This code defines a Customer model in the App\Models namespace in a PHP web application using the Laravel framework. The model is used to represent a customer in the application's database.

The model uses the HasFactory trait, which provides a way to create model instances using a factory. A factory is a class that generates fake data for testing purposes.

The \$fillable property is an array of attributes that are mass assignable. This means that they can be set in a single statement, such as during the creation of a new model instance. In this case, the name, cnic, gender, and phone attributes can be mass assigned.

The model extends the base Model class provided by Laravel, which provides a number of convenient methods for interacting with a database table. For example, you can use the model to retrieve, update, and delete records from the table.

Controllers:

```

<?php

namespace App\Http\Controllers;

use App\Models\Customer;
use App\Http\Requests\StoreCustomerRequest;
use App\Http\Requests\UpdateCustomerRequest;

class CustomerController extends Controller
{
    /**
     * Display a listing of the resource.
     *
     * @return \Illuminate\Http\Response
     */
    public function index()
    {
        return view("Customers.index")->with("abc",Customer::all());
    }

    /**

```



```

* Show the form for creating a new resource.
*
* @return \Illuminate\Http\Response
*/
public function create()
{
    return view("Customers.create");
}

/**
 * Store a newly created resource in storage.
 *
 * @param \App\Http\Requests\StoreCustomerRequest $request
 * @return \Illuminate\Http\Response
 */
public function store(StoreCustomerRequest $request)
{
    Customer::Create($request->all());
    return redirect()->route('customers.index')
        ->with('success','customer added successfully.');
```

```

}

/**
 * Display the specified resource.
 *
 * @param \App\Models\Customer $customer
 * @return \Illuminate\Http\Response
 */
public function show(Customer $customer)
{
    return view("Customers.show")->with("customer",$customer);
}

/**
 * Show the form for editing the specified resource.
 *
 * @param \App\Models\Customer $customer
 * @return \Illuminate\Http\Response
 */
public function edit(Customer $customer)
{
    return view("Customers.edit")->with("customer",$customer);
}

/**
 * Update the specified resource in storage.
 *
 * @param \App\Http\Requests\UpdateCustomerRequest $request
```

```

    * @param \App\Models\Customer $customer
    * @return \Illuminate\Http\Response
    */
    public function update(UpdateCustomerRequest $request, Customer $customer)
    {
        $customer->name=$request->name;
        $customer->gender=$request->gender;
        $customer->cnic=$request->cnic;
        $customer->phone=$request->phone;
        $customer->save();
        return redirect()->route('customers.index')
            ->with('success','customer Updated successfully.');
```



```

    /**
     * Remove the specified resource from storage.
     *
     * @param \App\Models\Customer $customer
     * @return \Illuminate\Http\Response
     */
    public function destroy(Customer $customer)
    {
        $customer->delete();
        return redirect()->route('customers.index')
            ->with('success','customer deleted successfully.');
```

This code defines a CustomerController class in the App\Http\Controllers namespace in a PHP web application using the Laravel framework. The controller is responsible for handling HTTP requests related to the customer resource, such as creating, updating, and deleting customers.

The controller contains several methods, each of which corresponds to a specific HTTP request method and URL. For example, the index method handles a GET request to the /customers URL and is responsible for displaying a list of all customers.

The create, store, edit, and update methods handle requests related to creating and updating customers, while the show and destroy methods handle requests related to viewing and deleting customers, respectively.

The \$request parameter in the store, update methods is an instance of a request class that represents the incoming HTTP request. These request classes, StoreCustomerRequest and UpdateCustomerRequest, are used to validate the incoming request data.

The `$customer` parameter in the show, edit, update, and destroy methods is an instance of the Customer model and represents the customer being accessed or modified.

Finally, the controller uses views to render HTML responses for each of the methods. The views are located in the `resources/views/Customers` directory and use blade templates, which are a simple, yet powerful templating language provided by Laravel.

.env

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=here your database name(blog)
DB_USERNAME=here database username(root)
DB_PASSWORD=here database password(root)
```

Resources/view/Customers:
create.blade.php:

```
@extends('Masterlayout.layout')

@section("mainContent")

<div class="parent">
    <div class="subparent">
        <form action="{{ route('customers.store') }}" method="post">
            <div class="mb-3 mt-3" style="width: 100%">
                <label for="name" class="form-label">Enter Customer Name:</label>
                <input type="text" class="form-control" id="name" placeholder="Enter Customer
Name"
                    name="name">
            </div>
            @csrf
            <div class="mb-3 mt-3" style="width: 100%">
                <label for="cnic" class="form-label">Enter Customer CNIC:</label>
                <input type="cnic" class="form-control" id="cnic"
                    placeholder="Enter Customer cnic" name="cnic">
            </div>
            <div class="mb-3">
                <label for="gender" class="form-label">Select Customer gender:</label>
```

```

        <div class="form-check">
            <input class="form-check-input" type="radio" name="gender"
id="flexRadioDefault1" value="male" checked>
            <label class="form-check-label" for="flexRadioDefault1">
                Male
            </label>
        </div>
        <div class="form-check">
            <input class="form-check-input" type="radio" name="gender"
id="flexRadioDefault2" value="female">
            <label class="form-check-label" for="flexRadioDefault2">
                Female
            </label>
        </div>

        <div class="mb-3 mt-3" style="width:100%">
            <label for="phone" class="form-label">Enter Customer Phone:</label>
            <input type="text" class="form-control" id="phone"
                placeholder="Enter Customer phone" name="phone">
        </div>

        <button type="submit" value="Save" class="btn btn-primary"
            style="width: 100%; background-color: rgb(46, 166, 46);border:2px rgb(46, 166,
46)">Submit</button>
        </form>
    </div>
</div>
@endsection

```

This code is a blade template file in a PHP web application using the Laravel framework. Blade is a simple, yet powerful templating language provided by Laravel that allows you to mix PHP and HTML in your views. The `@extends` directive is used to specify that this template should extend a layout template located in the `resources/views/Masterlayout` directory and named `layout.blade.php`. The layout template defines the overall structure of the page, such as the header, footer, and sidebars, and the content section will be replaced with the content of this template.

The `@section` directive defines a section of content that can be inserted into the layout template. In this case, the `mainContent` section is defined, which will be inserted into the layout template in the place where the `@yield('mainContent')` directive appears.

The template contains an HTML form with a number of input fields and a submit button. The form action is set to the `customers.store` route, which will be handled by the `store` method of the `CustomerController` class. The method will be called when the form is submitted using the POST method.

The `@csrf` directive is used to insert a hidden input field containing a CSRF (Cross-Site Request Forgery) token into the form. This token is used to protect the application from CSRF attacks, which are a type of malicious exploit that can occur when a user's web browser is tricked into making a request to a website that the user did not intentionally initiate.

The template also contains several input elements and a group of radio buttons for entering customer data and selecting the customer's gender.

Finally, the template contains a submit button, which will submit the form when clicked. When the form is submitted, the customer data will be sent to the server, and the `store` method of the `CustomerController` will be called to handle the request.

LAB 12: Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine (Part 2)

Objectives:

- Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine.

Theoretical Description:

Lab Task(s):

TASK 1: Use Laravel code first approach to create CRUD application using MySQL and Blade templating engine.

Solution:

edit.blade.php:

```
@extends('Masterlayout.layout')

@section("mainContent")

<div class="parent">
    <div class="subparent">
        <form action="{{ route('customers.update', ['customer'=>$customer->id]) }}"
method="post" class="w-50 mx-auto">
            @csrf
            @method("put")

            <div class="mb-3 mt-3" style="width: 100%">
                <label for="name" class="form-label">Enter Customer Name:</label>
                <input type="text" class="form-control" id="name" placeholder="Enter Customer
Name"
                    name="name"
                    value="{{ $customer->name }}">
            </div>
            @csrf
            <div class="mb-3 mt-3" style="width: 100%">
                <label for="cnic" class="form-label">Enter Customer CNIC:</label>
                <input type="cnic" class="form-control" id="cnic"
                    placeholder="Enter Customer CNIC" name="cnic" value="{{ $customer->cnic }}">
            </div>
            <div class="mb-3">
                <div class="form-check">
                    <input class="form-check-input" type="radio" name="gender"
id="flexRadioDefault1" value="male" {{ $customer->gender == "male" ? "Checked":"" }}>
```

```

        <label class="form-check-label" for="flexRadioDefault1">
            Male
        </label>
    </div>
    <div class="form-check">
        <input class="form-check-input" type="radio" name="gender"
id="flexRadioDefault2" value="female" {{ $customer->gender == "female" ? "Checked":"" }}>
        <label class="form-check-label" for="flexRadioDefault2">
            Female
        </label>
    </div>
</div>

<div class="mb-3 mt-3" style="width:100%">
    <label for="phone" class="form-label">Enter Customer Phone:</label>
    <input type="text" class="form-control" id="phone"
        placeholder="Enter Customer Phone" name="phone" value="{{ $customer->phone }}">
</div>

    <button type="submit" value="Save" class="btn btn-primary"
        style="width: 100%; background-color: rgb(46, 166, 46);border:2px rgb(46, 166, 46)">Submit</button>
</form>
</div>
</div>
@endsection

```

This code is a blade template file in a PHP web application using the Laravel framework. Blade is a simple, yet powerful templating language provided by Laravel that allows you to mix PHP and HTML in your views. The **@extends** directive is used to specify that this template should extend a layout template located in the **resources/views/Masterlayout** directory and named **layout.blade.php**. The layout template defines the overall structure of the page, such as the header, footer, and sidebars, and the content section will be replaced with the content of this template.

The **@section** directive defines a section of content that can be inserted into the layout template. In this case, the **mainContent** section is defined, which will be inserted into the layout template in the place where the **@yield('mainContent')** directive appears.

The template contains an HTML form with a number of input fields and a submit button. The form action is set to the **customers.update** route, which will be handled by the **update** method of the **CustomerController** class. The method will be called when the form is submitted using the **PUT** method.

The **@csrf** directive is used to insert a hidden input field containing a CSRF (Cross-Site Request Forgery) token into the form. This token is used to protect the application from CSRF attacks, which are a type of malicious exploit that can occur when a user's web browser is tricked into making a request to a website that the user did not intentionally initiate.

The **@method** directive is used to specify that the form should be submitted using the **PUT** HTTP method. This is necessary because HTML forms only support the **GET** and **POST** methods, and the **update** method expects a **PUT** request.

The template also contains several **input** elements and a group of **radio** buttons for entering customer data and selecting the customer's gender. The values of the input fields and radio buttons are pre-populated with the current values of the customer being edited, which is passed to the template as the **\$customer** variable. Finally, the template contains a submit button, which will submit the form when clicked. When the form is submitted, the updated customer data will be sent to the server, and the **update** method of the **CustomerController** will be called to handle the request.

index.blade.php:

```
@extends('Masterlayout.layout')

@section("mainContent")

<div class="container">
    <a href="{{ route('customers.create') }}" class="btn btn-primary m-3">Add a new
customer</a>
    <table class="table w-75 mx-auto mt-5">
        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>CNIC</th>
            <th>Gender</th>
            <th>Phone</th>
            <th>Actions</th>
        </tr>
        @foreach($abc as $customer)
            <tr>
                <td>{{ $customer->id }}</td>
                <td>{{ $customer->name }}</td>
                <td>{{ $customer->cnic }}</td>
                <td>{{ $customer->gender }}</td>
                <td>{{ $customer->phone }}</td>
                <td>
```



```

                <a href="{{ route('customers.show', ['customer'=>$customer->id]) }}"
class="btn btn-success btn-sm"><i class="fa-sharp fa-solid fa-eye"></i></a> |
                <a href="{{ route('customers.edit', ['customer'=>$customer->id]) }}"
class="btn btn-warning btn-sm"><i class="fa-solid fa-pen-to-square text-light"></i></a> |
                <form action="{{ route('customers.destroy', ['customer'=>$customer->id])
}}" style="display: inline" method="POST">
                    @csrf
                    @method("delete")
                    <button type="submit" class="btn btn-danger btn-sm"><i class="fa-solid
fa-trash"></i></i></button>
                </form>
            </td>
        </tr>
    @endforeach
</table>
</div>

@endsection

```

This code is a blade template file in a PHP web application using the Laravel framework. Blade is a simple, yet powerful templating language provided by Laravel that allows you to mix PHP and HTML in your views. The **@extends** directive is used to specify that this template should extend a layout template located in the **resources/views/Masterlayout** directory and named **layout.blade.php**. The layout template defines the overall structure of the page, such as the header, footer, and sidebars, and the content section will be replaced with the content of this template.

The **@section** directive defines a section of content that can be inserted into the layout template. In this case, the **mainContent** section is defined, which will be inserted into the layout template in the place where the **@yield('mainContent')** directive appears.

The template contains an HTML table with a number of columns for displaying customer data, such as the customer's name, CNIC, gender, and phone number. The table also contains an "Actions" column with buttons for viewing, editing, and deleting customers.

The template also contains an "Add a new customer" button, which will redirect the user to the `customers

Show.blade.php:

```

@extends('Masterlayout.layout')

@section("mainContent")
<div class="container">
    <div class="card">

```

```

<div class="card-header">
  <div class="card-title">
    {{$customer->name}}
  </div>
</div>
<div class="card-body">
  <p><strong>CNIC: </strong> {{$customer->cnic}}</p>
  <p><strong>Gender: </strong> {{$customer->gender}}</p>
  <p><strong>Phone: </strong>{{$customer->phone}}</p>

</div>
</div>
</div>
@endsection

```

This code is a blade template file in a PHP web application using the Laravel framework. Blade is a simple, yet powerful templating language provided by Laravel that allows you to mix PHP and HTML in your views. The **@extends** directive is used to specify that this template should extend a layout template located in the **resources/views/Masterlayout** directory and named **layout.blade.php**. The layout template defines the overall structure of the page, such as the header, footer, and sidebars, and the content section will be replaced with the content of this template.

The **@section** directive defines a section of content that can be inserted into the layout template. In this case, the **mainContent** section is defined, which will be inserted into the layout template in the place where the **@yield('mainContent')** directive appears.

The template contains an HTML card element with a header and body. The header contains the customer's name and the body contains the customer's CNIC, gender, and phone number. The customer data is displayed using the **\$customer** variable, which is passed to the template by the **CustomerController**.

The template also contains an "Actions" column with buttons for viewing, editing, and deleting customers. The template displays the customer's name, CNIC, gender, and phone number in the body of the card.

MasterLayout/layout: Layout.blade.php:

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Pharmacy Managenment System</title>

```

```

    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css"
rel="stylesheet"
        integrity="sha384-rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Kadq2F9CUG65"
crossorigin="anonymous">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.2.1/css/all.min.css">

    <style>
        body{
            background: lightyellow;
            margin: 20px;
            padding: 20px;
        }
        .parent {
            /* border: 2px solid blue; */
            height: 100vh;
            display: flex;
            align-items: center;
            justify-content: center;
            background: url("/pbackground.jpg");
            background-size: cover;
        }
        label {
            color: green;
        }
        .subparent {
            width: 50%;
            /* border: 2px solid green; */
        }

    </style>
</head>
<body>
    @yield('mainContent')
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-kenU1KFdBIe4zVF0s0G1M5b4hcxpyD9F7jL+jjXkk+Q2h455rYXK/7HAuoJl+0I4"
crossorigin="anonymous"></script>
    </body>
</html>

```

Route/web.php:

```
<?php
```

```

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\CustomerController;
Route::resource('customers', CustomerController::class); ?>

```

This code is a route definition in a PHP web application using the Laravel framework. It defines a resource route for the **customers** resource, which maps HTTP verbs to controller actions.

The **Route::resource** method creates a number of routes for the **customers** resource that map to the actions of the **CustomerController** controller. These routes include:

- **GET /customers:** Maps to the **index** action of the **CustomerController**, which displays a list of customers.
- **GET /customers/create:** Maps to the **create** action of the **CustomerController**, which displays a form for creating a new customer.
- **POST /customers:** Maps to the **store** action of the **CustomerController**, which stores a new customer in the database.
- **GET /customers/{customer}:** Maps to the **show** action of the **CustomerController**, which displays a specific customer.
- **GET /customers/{customer}/edit:** Maps to the **edit** action of the **CustomerController**, which displays a form for editing a specific customer.
- **PUT /customers/{customer}:** Maps to the **update** action of the **CustomerController**, which updates a specific customer in the database.
- **DELETE /customers/{customer}:** Maps to the **destroy** action of the **CustomerController**, which deletes a specific customer from the database.

These routes allow you to perform all of the CRUD (create, read, update, delete) operations on the **customers** resource.

Now we are ready to run our crud application example with laravel 9 so run bellow command for quick run:

```
php artisan serve
```

LAB 13: Create RESTful APIs for tasks table having columns id, text, day and reminder using Laravel

Objectives:

- Learning the use of Laravel to consume RESTful APIs for tabular structure.

Theoretical Description:

Lab Task(s):

TASK 1: Create RESTful APIs for tasks table having columns id, text, day and reminder using Laravel.

Solution:

Migrations:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateTasksTable extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('tasks', function (Blueprint $table) {
            $table->id();
            $table->string("text");
            $table->string("day");
            $table->string("reminder");

            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('tasks');
    }
}
```

This code is defining a migration class in PHP using the Laravel framework. Migrations are a way to manage changes to the database schema of a project. They allow developers to create and modify the database schema in a versioned and reversible way.

The `CreateTasksTable` class extends the `Migration` class and defines two methods: `up` and `down`. The `up` method is used to specify the changes that should be made to the database schema when the migration is run, and the `down` method is used to specify the changes that should be made to reverse the changes made by the `up` method.

In this case, the `up` method is using the Laravel Schema builder to create a new database table called `tasks`, with four columns: an `id` column, a `text` column, a `day` column, and a `reminder` column. The `id` column is an auto-incrementing primary key, and the `timestamps` method adds `created_at` and `updated_at` columns to the table.

The `down` method simply drops the `tasks` table if it exists.

To run this migration and apply the changes to the database, you would use the Artisan command line interface that comes with Laravel: `php artisan migrate`. To reverse the changes and undo the migration, you would use the `rollback` command: `php artisan migrate:rollback`.

Models:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
    use HasFactory;
    protected $fillable = [

        'text', 'day', 'reminder',

    ];
}
```

This code is defining a model class in PHP using the Laravel framework. Models are used to represent the data stored in the database, and they provide a convenient way to query and manipulate that data.

The Task class extends the base Model class and uses the HasFactory trait. The HasFactory trait allows you to use Laravel's built-in model factories to generate fake test data for your application.

The \$fillable property is an array of attributes that are mass assignable. This means that you can use the create method on the model to create a new instance of the Task model and set the values of the text, day, and reminder attributes in a single step.

Kernal.php: **Add this code :**

```
'api' => [
    \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,
    'throttle:api',
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
],
```

This code is defining a middleware group in the Laravel framework. Middleware are pieces of code that can be executed before or after a request is handled by your application. They can be used to perform various tasks such as authentication, rate limiting, or modifying the request or response.

In this case, the middleware group is called api and it consists of three middleware classes: EnsureFrontendRequestsAreStateful, throttle, and SubstituteBindings.

The EnsureFrontendRequestsAreStateful middleware is part of the Laravel Sanctum package and it is used to ensure that requests coming from a Single Page Application (SPA) are stateful. This means that the middleware will check if the request contains a valid token (such as a CSRF token) and if it does not, it will redirect the user to the login page.

The throttle middleware is a built-in Laravel middleware that is used to rate limit requests to your application. It can be configured to allow a certain number of requests per minute or per second from a given IP address. The SubstituteBindings middleware is also a built-in Laravel middleware that is used to substitute route model bindings in the request. This means that if you have defined a route parameter as a model ID, the middleware will automatically retrieve the model instance from the database and bind it to the request.

Controllers\BaseController.php:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class BaseController extends Controller
{
    public function sendResponse($result, $message)
    {
        $response = [
            'success' => true,
            'data'     => $result,
            'message'  => $message,
        ];
        return response()->json($response, 200);
    }
    public function sendError($error, $errorMessagees = [], $code = 404)
    {
        $response = [
            'success' => false,
            'message' => $error,
        ];
        if(!empty($errorMessagees)){
            $response['data'] = $errorMessagees;
        }
        return response()->json($response, $code);
    }
}
```

This code is defining a base controller class in PHP using the Laravel framework. The base controller is a class that provides utility methods that can be shared by multiple controllers in your application.

The base controller extends the built-in Controller class provided by Laravel and defines two methods: sendResponse and sendError.

The sendResponse method is used to send a JSON response back to the client with a success status and a message. It takes two arguments: \$result, which is the data to be returned to the client, and \$message, which is a string message to be included in the response. The method creates an associative array with three keys: success, data, and message, and then returns a JSON response using the response() helper function provided by Laravel.

The `sendError` method is used to send a JSON response back to the client with an error status and a message. It takes three arguments: `$error`, which is the error message to be included in the response, `$errorMessages`, which is an optional array of additional error messages, and `$code`, which is the HTTP status code to be returned. The method creates an associative array with two keys: `success` and `message`, and then returns a JSON response using the `response()` helper function. If the `$errorMessages` argument is provided, it is added to the response array as the `data` key.

RegisterController.php:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Http\Controllers\BaseController as BaseController;
use App\Models\User;

use Illuminate\Support\Facades\Auth;
use Validator;

class RegisterController extends BaseController
{
    public function register(Request $request)
    {
        //Validation
        $validator = Validator::make($request->all(), [
            'name' => 'required',
            'email' => 'required|email',
            'password' => 'required',
            'c_password' => 'required|same:password',
        ]);
        if($validator->fails()){
            return $this->sendError('Validation Error.', $validator->errors());
        }
        //Picking data from request and store in input to further processing
        $input = $request->all();
        //Encrypting data
        $input['password'] = bcrypt($input['password']);
        $user = User::create($input); //Create a new user
        $success['token'] = $user->createToken('MyApp')->plainTextToken; //Generating a token
        $success['name'] = $user->name;
        return $this->sendResponse($success, 'User register successfully.');
```

```
    }

    public function login(Request $request)
    {
        if(Auth::attempt(['email' => $request->email, 'password' => $request->password])){
            $user = Auth::user();
            $success['token'] = $user->createToken('MyApp')->plainTextToken;
```

```

        $success['name'] = $user->name;
        return $this->sendResponse($success, 'User login successfully.');
```

```

    } else {
        return $this->sendError('Unauthorised.', ['error'=>'Unauthorised']);
    }
}
}
}

```

This code is defining a controller class in PHP using the Laravel framework. The controller is responsible for handling HTTP requests and returning responses to the client.

The controller extends the BaseController class defined in the previous code example, which provides utility methods for sending JSON responses to the client.

The controller defines two methods: register and login.

The register method is used to handle a registration request from the client. It first performs validation on the request data using the built-in Validator class. If the validation fails, it returns an error response using the sendError method inherited from the base controller. If the validation succeeds, it picks the data from the request, encrypts the password using the bcrypt function, and creates a new user in the database using the User model. Finally, it generates a token for the user using the createToken method provided by Laravel's built-in authentication system and returns a success response to the client.

The login method is used to handle a login request from the client. It uses the Auth facade and the attempt method to authenticate the user based on the email and password provided in the request. If the authentication succeeds, it retrieves the authenticated user, generates a token for the user using the createToken method, and returns a success response to the client. If the authentication fails, it returns an error response to the client.

These methods can be used to handle user registration and login requests in your application. For example, you could define routes that map to these methods and use them to handle requests to the /register and /login endpoints.

TaskController.php:

```

<?php

namespace App\Http\Controllers;

use App\Models\Task;
use App\Http\Controllers\BaseController as BaseController;
use Illuminate\Http\Request;
use Validator;
use App\Http\Resources\TaskResource;

```

```
class TaskController extends BaseController
{
public function index()

    {

        $task = Task::all();


        return $this->sendResponse(TaskResource::collection($task), 'Task retrieved
successfully.');
```

```
    }

public function store(Request $request)

    {

        $input = $request->all();


        $validator = Validator::make($input, [

            'text' => 'required',

            'day' => 'required',
            'reminder' => 'required'

        ]);


        if($validator->fails()){

            return $this->sendError('Validation Error.', $validator->errors());

        }


        $task = Task::create($input);


        return $this->sendResponse(new TaskResource($task), 'Task created successfully.');
```

```
    }
```

```
public function show($id)

{

    $task = Task::find($id);

    if (is_null($task)) {

        return $this->sendError('Product not found.');
```

```
    }

    return $this->sendResponse(new TaskResource($task), 'Task retrieved successfully.');
```

```
    }

    public function update(Request $request, Task $task)

    {

        $input = $request->all();

        $validator = Validator::make($input, [

            'text' => 'required',

            'day' => 'required',

            'reminder' => 'required'

        ]);

        if($validator->fails()){

            return $this->sendError('Validation Error.', $validator->errors());

        }

        $task->text = $input['text'];
```

```

        $task->day = $input['day'];
        $task->reminder = $input['reminder'];

        $task->save();

        return $this->sendResponse(new TaskResource($task), 'Task updated successfully.');
```

 }

 public function destroy(Task \$task)
 {
 \$task->delete();

 return \$this->sendResponse([], 'Task deleted successfully.');
 }
}

This code is defining a controller class in PHP using the Laravel framework. The controller is responsible for handling HTTP requests and returning responses to the client.

The controller extends the BaseController class defined in a previous code example, which provides utility methods for sending JSON responses to the client.

The controller defines several methods for managing tasks in the application:

- The index method is used to retrieve a list of all tasks from the database and return them to the client as a collection of TaskResource resources.
- The store method is used to create a new task in the database. It validates the request data, creates a new task using the Task model, and returns a response to the client with the new task data.
- The show method is used to retrieve a single task from the database based on its ID. If the task is not found, it returns an error response. Otherwise, it returns the task data to the client.
- The update method is used to update an existing task in the database. It validates the request data, updates the task with the new data, and returns a response to the client with the updated task data.

- The destroy method is used to delete a task from the database. It removes the task from the database and returns a response to the client indicating that the task was deleted successfully.

These methods can be used to handle requests to the /tasks endpoint in your application. For example, you could define routes that map to these methods and use them to handle requests to the /tasks endpoint to create, retrieve, update, and delete tasks.

TaskResources.php:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class TaskResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array|\Illuminate\Contracts\Support\Arrayable|\JsonSerializable
     */
    public function toArray($request)
    {
        return [

            'id' => $this->id,

            'text' => $this->text,

            'day' => $this->day,
            'reminder' => $this->reminder,

            'created_at' => $this->created_at->format('d/m/Y'),

            'updated_at' => $this->updated_at->format('d/m/Y'),

        ];
    }
}
```

This code is defining a resource class in PHP using the Laravel framework. Resources are used to transform complex data structures (such as model instances or collections) into simple arrays or JSON objects that can be returned to the client.

The TaskResource class extends the built-in JsonResource class provided by Laravel and defines a single method: toArray.

The toArray method is used to transform the resource instance into an array. It takes a single argument: \$request, which is an instance of the current HTTP request. The method returns an associative array with several keys:

- id: the ID of the task
- text: the text of the task
- day: the day of the task
- reminder: the reminder of the task
- created_at: the creation date of the task, formatted as a string
- updated_at: the update date of the task, formatted as a string

The resource class can be used in a controller to transform a model instance or collection into a resource before returning it to the client.

Route\api.php:

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\TaskController;

Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    return $request->user();
});

Route::controller(RegisterController::class)->group(function(){
    Route::post('register', 'register');
    Route::post('login', 'login');
});

Route::middleware('auth:sanctum')->group(function(){
    Route::resource('tasks', TaskController::class);
});
```

This code is defining a set of routes in PHP using the Laravel framework. Routes are used to map URLs to controller actions in your application.

The first route uses the `middleware` method to apply the `auth:sanctum` middleware to the route. This middleware checks whether the request is authenticated using Laravel's built-in Sanctum authentication system. If the request is authenticated, the route closure function is executed. Otherwise, the request is rejected and an error is returned.

The closure function returns the authenticated user object to the client as a JSON response.

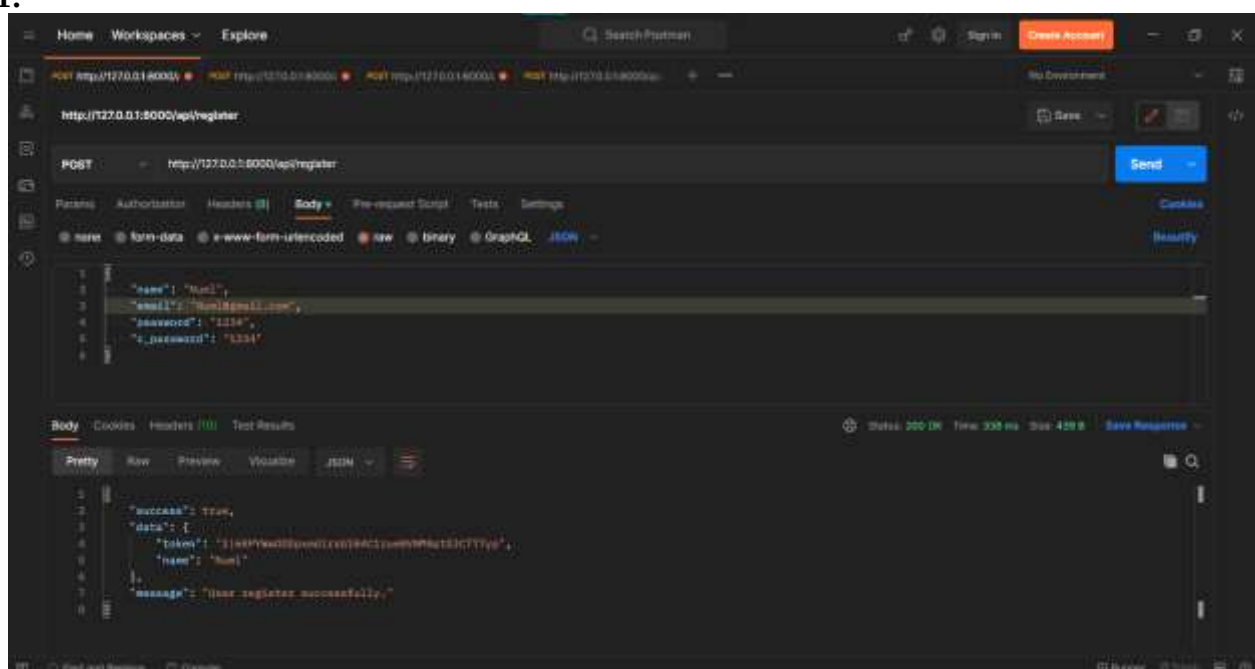
The second route group defines two routes for the register and login actions of the RegisterController controller. These routes are accessible to anyone and do not require authentication.

The third route group defines a resource route for the tasks resource. This route group applies the `auth:sanctum` middleware to all routes, meaning that they are only accessible to authenticated users. The resource route maps the standard HTTP verbs (GET, POST, PUT, DELETE) to the corresponding actions of the `TaskController` controller.

This configuration allows you to handle requests to the `/user`, `/register`, `/login`, and `/tasks` endpoints in your application. For example, a POST request to the `/tasks` endpoint would be handled by the `store` action of the `TaskController`.

After this download postman application on your pc and do accordingly as shown in pictures to see results:

Step1:



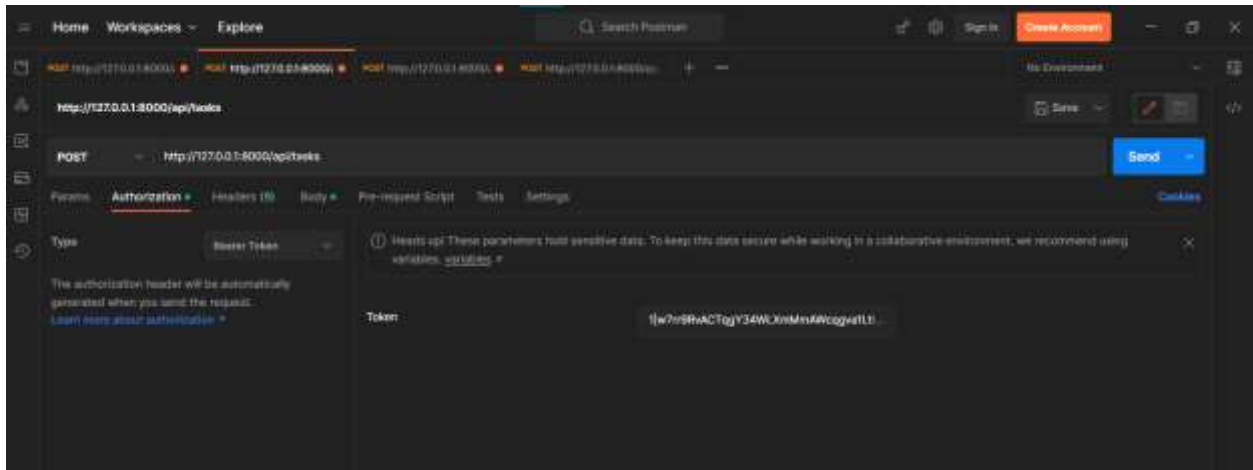
In this picture first I register the user

First add the url: <http://127.0.0.1:8000/api/register>

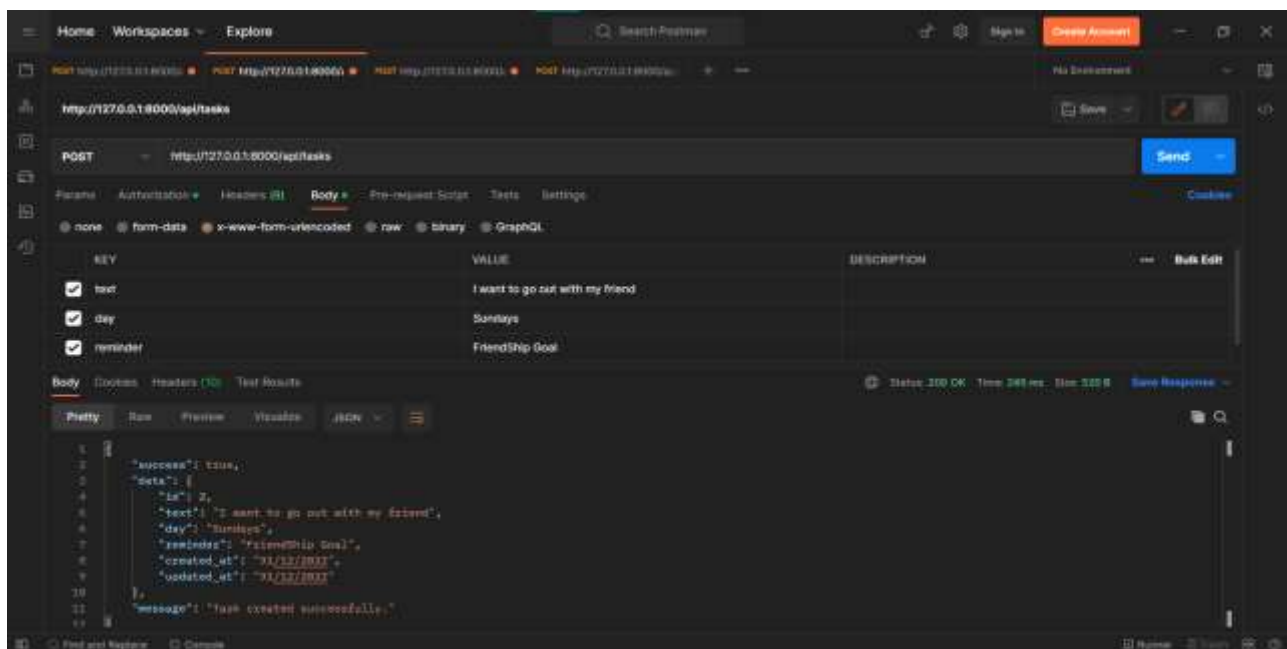
Method: Post

- IN the body add data in raw
- And after this click on send
- And you see the following results here copy the token we used it later

Step2:



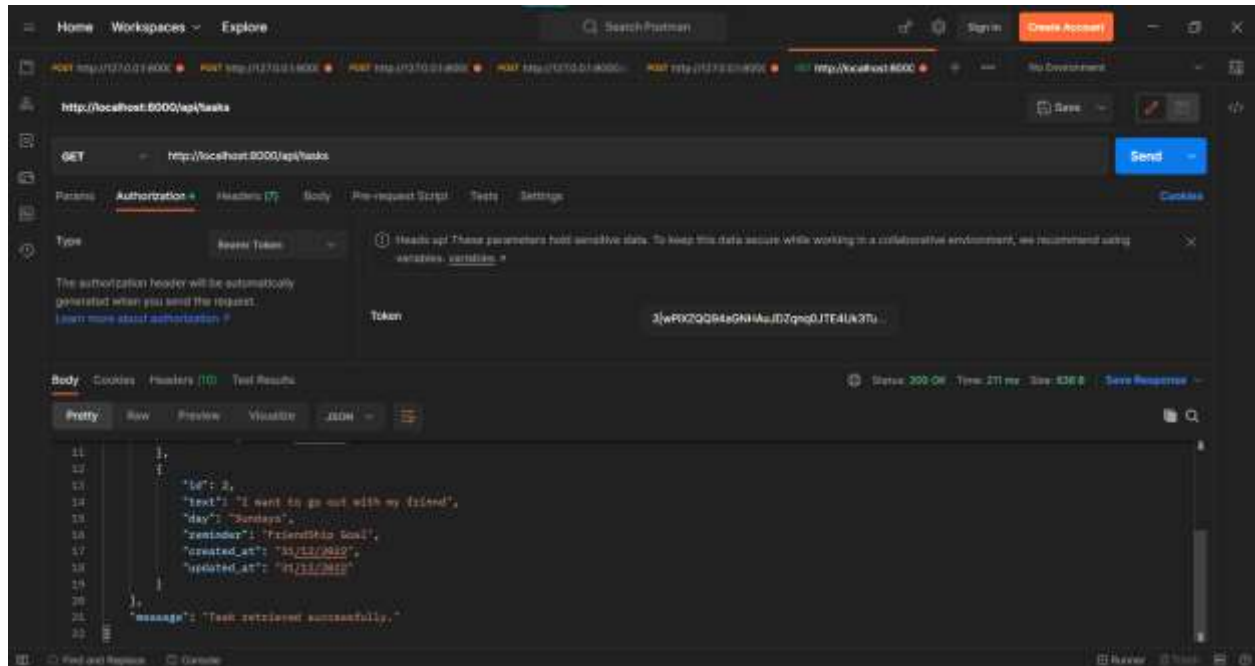
Add the following url: <http://127.0.0.1:8000/api/tasks>



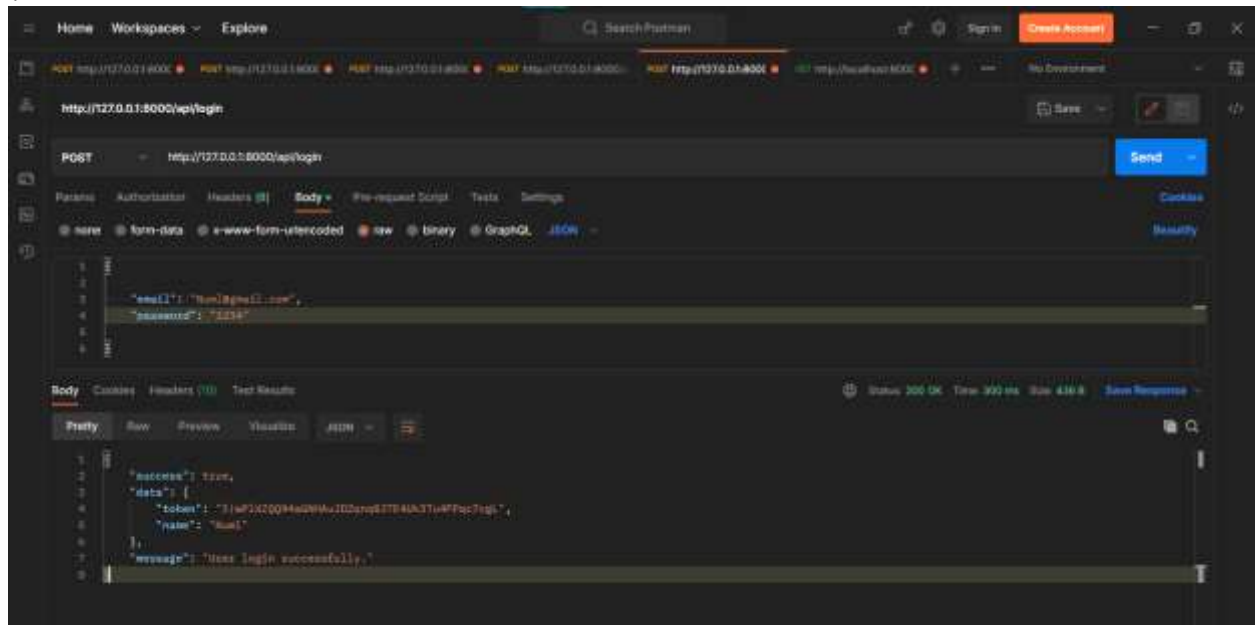
- Add method:POST
- Type Bear Token And Enter the following token

- Next click on body add data following
- After data added click on send and you see your data is entered

List shown:



Login:



LAB 14: Consuming GET API to display data in React application

Objectives:

- To develop environmental setup for ReactJS
- To understand the use of APIs for data display in react.

Theoretical Description:

ReactJS setting up Development Environment

To run any React application, we must have NodeJS installed on our PC. So, the very first step will be to install NodeJS.

Lab Task(s):

Step 1: Install NodeJS. You may visit the [official download link](#) of NodeJS to download and install the latest version of NodeJS. Once we have set up NodeJS on our PC, the next thing we need to do is set up React Boilerplate.

Step 2: Setting up react environment for older and latest versions, follow anyone according to your node version.

For Older Versions which include Node < 8.10 and npm < 5.6: Setting up React Boilerplate. We will install the boilerplate globally. Run the below command in your terminal or command prompt to install the React Boilerplate.

npm install -g create-react-app

After running the above command and successfully installing the boilerplate on your terminal will show some output as shown in the below image:



```
harsh@harsh-Lenovo-G50-30: ~  
harsh@harsh-Lenovo-G50-30:~$ sudo npm install -g create-react-app  
/usr/local/bin/create-react-app -> /usr/local/lib/node_modules/create-react-app/  
index.js  
+ create-react-app@1.5.2  
updated 1 package in 7.228s  
harsh@harsh-Lenovo-G50-30:~$
```

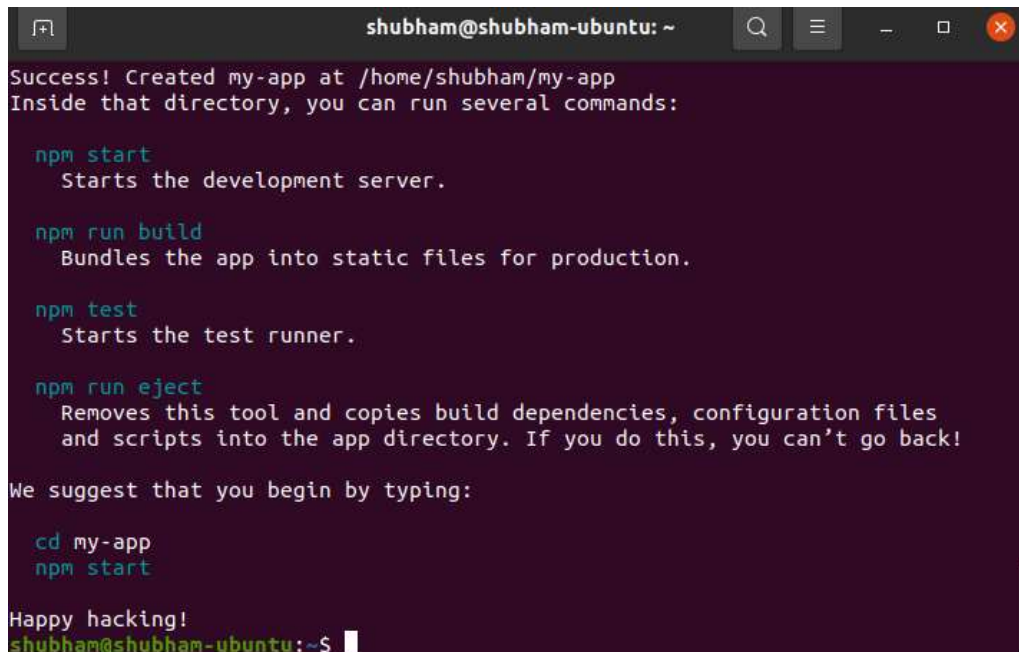
Now after successfully installing the boilerplate the next thing we will do is create our React app. We can use the create-react-app command for creating an app to React.

For the Latest Versions which include Node ≥ 8.10 and npm ≥ 5.6 : For using the latest features of JavaScript which provides a good development experience the machine should contain a version of Node ≥ 8.10 and npm ≥ 5.6 .

Run the below command to create a new project

npx create-react-app my-app

the above command will create the app name my-app as shown in the image below :



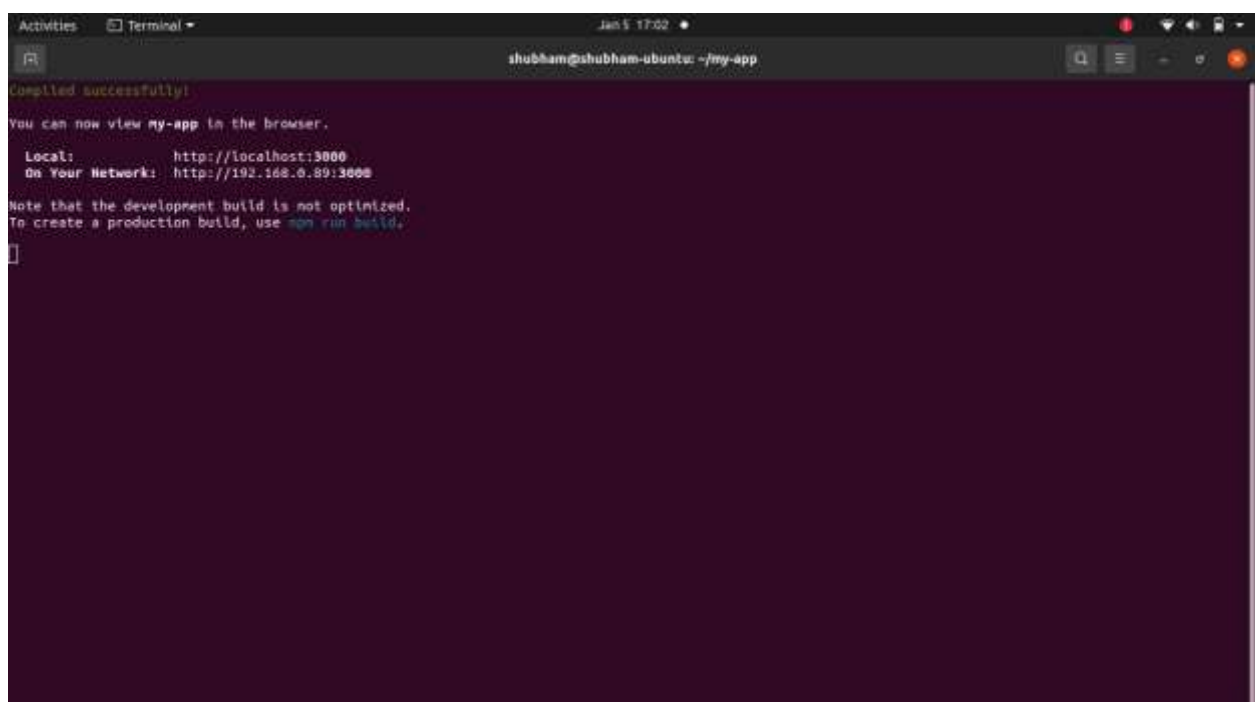
```
shubham@shubham-ubuntu: ~  
Success! Created my-app at /home/shubham/my-app  
Inside that directory, you can run several commands:  
  
npm start  
  Starts the development server.  
  
npm run build  
  Bundles the app into static files for production.  
  
npm test  
  Starts the test runner.  
  
npm run eject  
  Removes this tool and copies build dependencies, configuration files  
  and scripts into the app directory. If you do this, you can't go back!  
  
We suggest that you begin by typing:  
  
cd my-app  
npm start  
  
Happy hacking!  
shubham@shubham-ubuntu: ~$
```

You can run the project by typing the command `cd my-app`.

cd my-app

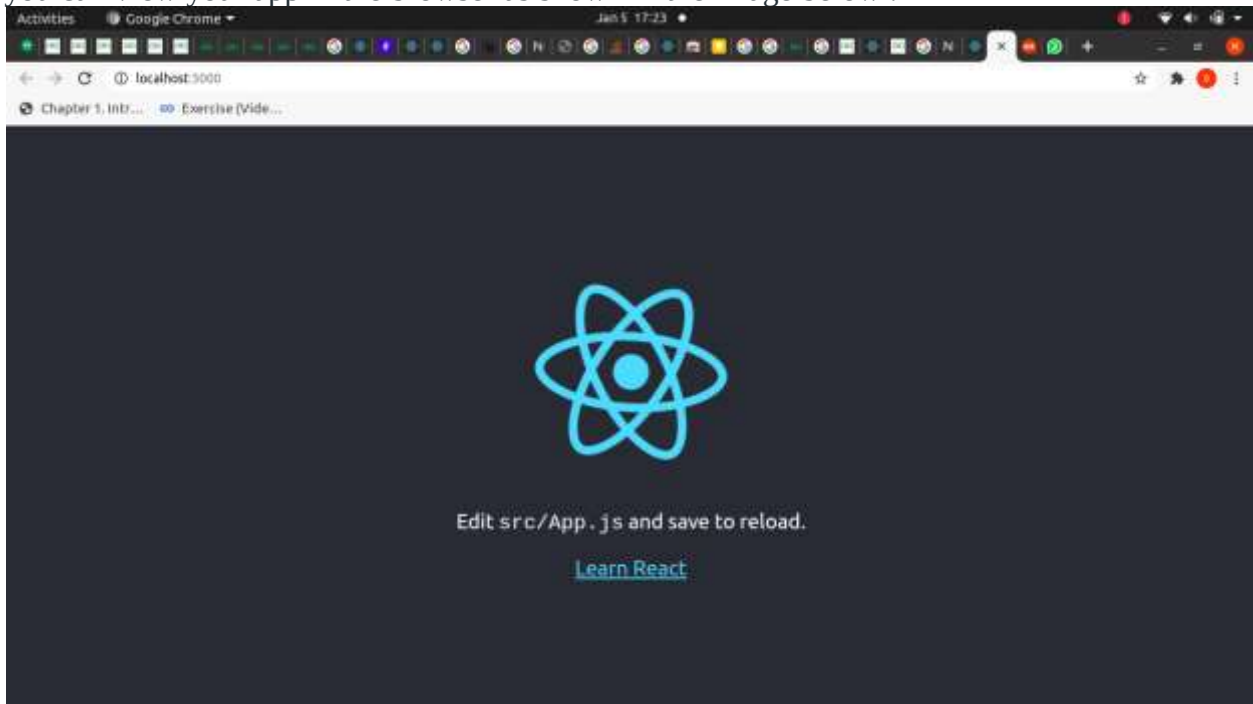
npm start

It will give you the output in the terminal as shown in the image below:



```
Jan 5 17:02  
shubham@shubham-ubuntu: ~/my-app  
Compiled successfully!  
  
You can now view my-app in the browser.  
  
Local:      http://localhost:3000  
On Your Network: http://192.168.0.89:3000  
  
Note that the development build is not optimized.  
To create a production build, use npm run build.  
[ ]
```

Now you can view your app in the browser as shown in the image below :

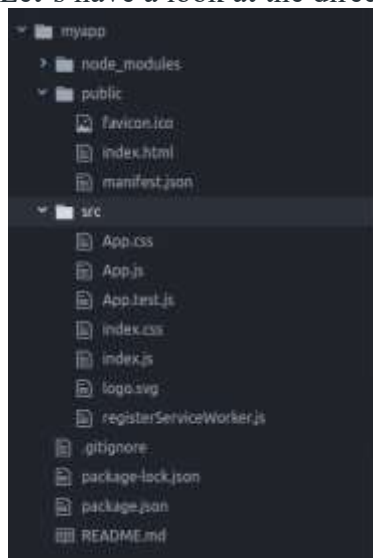


Step 3: Create a React app. Now to create an app we will use the boilerplate we installed. The below command will create an app named *myapp*.

create-react-app myapp

The above statement will create a new directory named *myapp* inside your current directory with a bunch of files needed to successfully run a React app.

Let's have a look at the directory created by the above command:



In the above directory, you can see a number of files. The main files we will be working on within the basic course are *index.html* and *index.js*. The *index.html* file will have a *div* element with *id* = "root", inside which everything will be rendered and all of our React code will be inside the *index.js* file.

Now, that we have successfully set up the development environment. The last thing left is to **start the development server**.

Step 4: Start the development server. To start the development server, go inside your current directory “myapp” and execute the below command:

npm start

On successfully running the above command your compiler will show the below message:

```
harsh@harsh-Lenovo-G50-30: ~/Desktop/myapp
Compiled successfully!

You can now view sampleapp in the browser.

Local:      http://localhost:3000/
On Your Network:  http://192.168.43.220:3000/

Note that the development build is not optimized.
To create a production build, use npm run build.
```

You can go to the URL shown in the above message to see the changes you are making in your App. By default, the above URL will show the below page:



That’s all! We have a development environment set up and ready. Now we will move ahead to start learning ReactJS development to make some use of it.

TASK : Use React and display of data from Api.

Solution:

Before make sure follow these steps:

To create a new, react app, you will need to have Node.js and npm (Node Package Manager) installed on your machine.

To create a new React app, follow these steps:

1. Open a terminal window and navigate to the directory where you want to create your new app.
2. Run the following command to create a new React app:

npx create-react-app my-app

This will create a new directory called **my-app** with a basic React app structure.

3. Navigate to the new app directory:

cd my-app

4. Start the development server:

npm start

This will start the development server and open a new browser window with your app. Any changes you make to the code will automatically be reflected in the browser.

5. To build the app for production, run the following command:

npm run build

This will create a production-ready build of your app in the **build** directory.

That's it! You have now created a new React app. You can start building your app by modifying the files in the **src** directory.

App.js:

```
import { useState, useEffect } from 'react';
import './App.css';
import Header from './components/Header';
import Products from './components/Products';

function App() {
  const [products, setProducts] = useState([])
  const url = "http://127.0.0.1:8000/api/products";
  const token = "1|fxbODCBWjuoy2lzkOVuI7PXyJH87WIERZKGFzON5";

  //Display all data from GET API
  const getOptions = {
    method:"GET",
    headers:{
      "Accept":"application/javascript",
      "Authorization":"Bearer "+ token
    },
    "redirect": 'follow'
  };

  useEffect(()=>{
    const getProducts = async () =>{
      const productsFromServer = await fetchProducts();
      setProducts(productsFromServer);
    }
    getProducts()
  },[]);

  const fetchProducts = async ()=>{
    const res = await fetch(url,getOptions);
    const data = await res.json();
  }
}
```

```

    return data.data;
  };
  body:JSON.stringify(product)
});
const data = await res.json();
setProducts([...products,data.data]);
}
}

export default App;

```

This code is a React component that represents a product management application. It uses the React hooks API, specifically the **useState** and **useEffect** hooks, to manage state and perform side effects.

The component has a state variable **products** that stores an array of products. It also has a constant **url** that stores the URL of an API that is used to fetch and manipulate the products. The **token** constant stores an authorization token that is used to authenticate the API requests.

The **useEffect** hook is used to perform a side effect when the component is rendered. It calls the **getProducts** function, which fetches the products from the API and sets them in the **products** state variable.

The **fetchProducts** function is an async function that uses the **fetch** function to make a GET request to the API and returns the data in the response.

Components Folder /Header.js:

```

import PropTypes from 'prop-types';
const Header=({title})=>{
  return (
    <>
      <h1>{title}</h1>
    </>
  );
}
Header.defaultProps={
  title:"React CRUD Application"
}
Header.propTypes={
  title:PropTypes.string.isRequired
}
export default Header;

```

This code is a React functional component that represents a header element. It uses the **PropTypes** library to define the expected types of the component's props.

The component has a single prop, **title**, which is passed as an argument to the function. It renders an **h1** element with the value of the **title** prop as its content.

The component also has a default prop, **title**, that is used if the **title** prop is not provided when the component is rendered. The default value is "React CRUD Application".

The component defines a **propTypes** object that specifies the expected type of the **title** prop. In this case, the **title** prop is required and must be a string.

The **PropTypes** library is used to validate the props of a component. If the props do not meet the defined criteria, the library will log a warning to the console. This can help identify mistakes in the component's usage and improve the reliability of the application.

Components Folder / **Product.js**:

```
import { trash3 } from "react-bootstrap-icons";
const Product = ({product, onDelete})=>{
  return (
    <tr>
      <td>{product.id}</td>
      <td>{product.name}</td>
      <td>{product.detail}</td>
      <td>
        <button onClick={()=>onDelete(product.id)} className="btn btn-sm btn-danger"><i
className="bi bi-trash3"></i></button>
      </td>
    </tr>
  );
}
export default Product;
```

This code is a React functional component that represents a table row element for displaying a product. It has two props: **product**, which is an object that represents a product, and **onDelete**, which is a function that is called when the delete button is clicked.

The component renders a **tr** element with four **td** elements that display the ID, name, and detail of the product, as well as a delete button. The delete button is a **button** element with an **onClick** event listener that calls the **onDelete** function and passes the product's ID as an argument. The **onDelete** function is expected to be a function that deletes the product from the database or removes it from the UI.

The component also includes an **i** element with a **className** attribute that sets the icon of the delete button. The **className** attribute refers to a specific icon in the **react-bootstrap-icons** library.

Components Folder /**Products.js**:

```
import Product from "../Product";
const Products = ({products,onDelete}) => {
  return (
    <table className="table">
      <thead>
        <tr>
          <th>Id</th>
          <th>Name</th>
          <th>Detail</th>
          <th>Action</th>
        </tr>
      </thead>
      <tbody>
        {
          products.length>0 && products.map(prod=>{
            return <Product key={prod.id} product={prod} onDelete={onDelete} />
          })
        }
      </tbody>
    </table>
  );
}

export default Products;
```

This code is a React functional component that represents a table for displaying a list of products. It has two props: **products**, which is an array of product objects, and **onDelete**, which is a function that is called when the delete button of a product is clicked.

The component renders a **table** element with a **thead** element for the table header and a **tbody** element for the table body. The table header has four **th** elements that display the headings for the ID, name, detail, and action columns.

The table body has a list of **Product** components, which are rendered using the **map** function. The **Product** components are passed the **onDelete** function as a prop and are rendered with the **product** prop set to each product in the **products** array.

The component also includes a conditional statement that checks if the **products** array is empty. If the array is empty, the component does not render any **Product** components.

LAB 15: Using POST and DELETE APIs in React application

Objectives:

- To learn data adding using API in react
- To understand data deletion in react using APIs

Theoretical Description:

Lab Task(s):

TASK : Use React to add and delete using API

Solution:

App.js:

```
import { useState, useEffect } from 'react';
import './App.css';
import Header from './components/Header';
import Products from './components/Products';
import AddProduct from './components/AddProduct';

function App() {
  const [products, setProducts] = useState([])
  const url = "http://127.0.0.1:8000/api/products";
  const token = "1|fxb0DCBWjuoy2lzkOVuI7PXyJH87WIERZKGFzON5";

  //Store a new Product
  const addProduct = async (product)=>{
    const res = await fetch(url,{
      method:"POST",
      headers:{
        "Accept":"application/javascript",
        "cache-control":"no-cache",
        "Content-Type":"application/json",
        "Authorization":"Bearer "+ token
      },
      "redirect": 'follow',
      body:JSON.stringify(product)
    });
    const data = await res.json();
    setProducts([...products,data.data]);
  }
}
```

```

//Delete Product
const deleteProduct = async (id)=>{
  const check = window.confirm("Are you sure you want to delete the record?");
  if(check){
    await fetch(`${url}/${id}`, {
      method: "DELETE",
      headers: {
        "Accept": "application/javascript",
        "cache-control": "no-cache",
        "Content-Type": "application/json",
        "Authorization": "Bearer " + token
      },
      "redirect": 'follow'
    });
    setProducts(products.filter(p=>p.id!==id));
  }
}

return (
  <div className="container">
    <Header text="some text here"/>
    <AddProduct addProduct={addProduct}/>
    <Products products={products} onDelete={deleteProduct}/>
  </div>
);
}

export default App;

```

This code is a React component that represents a product management application. It uses the React hooks API, specifically the **useState** and **useEffect** hooks, to manage state and perform side effects.

The component has a state variable **products** that stores an array of products. It also has a constant **url** that stores the URL of an API that is used to fetch and manipulate the products. The **token** constant stores an authorization token that is used to authenticate the API requests.

The **addProduct** function is an async function that adds a new product to the API using a POST request. It takes a product object as an argument and sends it in the body of the request. When the request is successful, it updates the **products** state variable by adding the new product to the array.

The **deleteProduct** function is an async function that deletes a product from the API using a DELETE request. It takes the ID of the product as an argument and sends it as part of the URL of the request. When the request is successful, it updates the **products** state variable by filtering out the product with the matching ID.

Components Folder /AddProduct.js:

```
import { useState } from "react";

const AddProduct=({addProduct})=>{
  const [name, setName]=useState('');
  const [detail, setDetail]=useState('');
  const changeInDetail = (e)=>{
    console.log(e.target.value);
    setDetail(e.target.value);
  }
  const onSubmit = (e)=>{
    e.preventDefault();
    if(!name){
      alert("Please enter name");
      return;
    }
    if(!detail){
      alert("Please enter product detail");
      return;
    }
    addProduct({name,detail});
    setName('');
    setDetail('');
  }
  return (
    <form onSubmit={onSubmit}>
      <div className="mb-3">
        <label className="form-label">Name</label>
        <input type="text" className="form-control" placeholder="Enter Product Name"
value={name} onChange={e=>setName(e.target.value)}/>
      </div>
      <div className="mb-3">
        <label htmlFor="detail" className="form-label">Detail</label>
        <textarea id="detail" name="detail" className="form-control" rows="3" cols="3"
value={detail} onChange={e=>setDetail(e.target.value)}></textarea>
      </div>
      <div className="mb-3">
        <input type="submit" className="btn btn-primary form-control" value="Save"/>
      </div>
    </form>
  )
}
export default AddProduct;
```

This code is a React functional component that represents a form for adding a new product. It uses the **useState** hook to manage state for the name and detail of the product.

The component has two state variables, **name** and **detail**, which are initialized to empty strings. It has two functions, **changeInDetail** and **onSubmit**, that are used to handle changes in the form inputs and submit the form, respectively.

The **changeInDetail** function is called when the value of the detail textarea is changed. It logs the new value to the console and updates the **detail** state variable with the new value.

The **onSubmit** function is called when the form is submitted. It prevents the default form submission behavior and checks if the name and detail fields are empty. If either field is empty, it displays an alert asking the user to enter a value. If both fields are filled, it calls the **addProduct** function that was passed as a prop and clears the form inputs.

The component renders a form element with three form groups: one for the name input, one for the detail textarea, and one for the submit button. Each form group has an event listener that calls the appropriate function when the value of the input is changed or the form is submitted. The form also displays the current values of the **name** and **detail** state variables.

Components Folder /Header.js:

```
import PropTypes from 'prop-types';
const Header=({title})=>{
  return (
    <>
      <h1>{title}</h1>
    </>
  );
}
Header.defaultProps={
  title:"React CRUD Application"
}
Header.propTypes={
  title:PropTypes.string.isRequired
}
export default Header;
```

This code is a React functional component that represents a header element. It uses the **PropTypes** library to define the expected types of the component's props.

The component has a single prop, **title**, which is passed as an argument to the function. It renders an **h1** element with the value of the **title** prop as its content.

The component also has a default prop, **title**, that is used if the **title** prop is not provided when the component is rendered. The default value is "React CRUD Application".

The component defines a **propTypes** object that specifies the expected type of the **title** prop. In this case, the **title** prop is required and must be a string.

The **PropTypes** library is used to validate the props of a component. If the props do not meet the defined criteria, the library will log a warning to the console. This can help identify mistakes in the component's usage and improve the reliability of the application.

Components Folder / **Product.js**:

```
import { trash3 } from "react-bootstrap-icons";
const Product = ({product, onDelete})=>{
  return (
    <tr>
      <td>{product.id}</td>
      <td>{product.name}</td>
      <td>{product.detail}</td>
      <td>
        <button onClick={()=>onDelete(product.id)} className="btn btn-sm btn-danger"><i
className="bi bi-trash3"></i></button>
      </td>
    </tr>
  );
}
export default Product;
```

This code is a React functional component that represents a table row element for displaying a product. It has two props: **product**, which is an object that represents a product, and **onDelete**, which is a function that is called when the delete button is clicked.

The component renders a **tr** element with four **td** elements that display the ID, name, and detail of the product, as well as a delete button. The delete button is a **button** element with an **onClick** event listener that calls the **onDelete** function and passes the product's ID as an argument. The **onDelete** function is expected to be a function that deletes the product from the database or removes it from the UI.

The component also includes an **i** element with a **className** attribute that sets the icon of the delete button. The **className** attribute refers to a specific icon in the **react-bootstrap-icons** library.

Components Folder /**Products.js**:

```
import Product from "../Product";
const Products = ({products,onDelete}) => {
  return (
    <table className="table">
      <thead>
        <tr>
          <th>Id</th>
          <th>Name</th>
          <th>Detail</th>
          <th>Action</th>
        </tr>
      </thead>
      <tbody>
        {
          products.length>0 && products.map(prod=>{
            return <Product key={prod.id} product={prod} onDelete={onDelete} />
          })
        }
      </tbody>
    </table>
  );
}

export default Products;
```

This code is a React functional component that represents a table for displaying a list of products. It has two props: **products**, which is an array of product objects, and **onDelete**, which is a function that is called when the delete button of a product is clicked.

The component renders a **table** element with a **thead** element for the table header and a **tbody** element for the table body. The table header has four **th** elements that display the headings for the ID, name, detail, and action columns.

The table body has a list of **Product** components, which are rendered using the **map** function. The **Product** components are passed the **onDelete** function as a prop and are rendered with the **product** prop set to each product in the **products** array.

The component also includes a conditional statement that checks if the **products** array is empty. If the array is empty, the component does not render any **Product** components.

LAB 16: Final-term Evaluation